

PreOne Prisma Schema v3.0

Prisma Schema — Enterprise Preschool Operating System

Document Version: 3.0

Status: Database Implementation Freeze

Date: 2026-07-14

Product: PreOne — Enterprise Preschool Operating System

Predecessor: DDD v1.0, ERD v3.0, ADR v3.0

Successor: NestJS Backend, Prisma Client, Database Migration, Seed Scripts

Classification: Internal Engineering Reference

Prepared by: PreOne Architecture & Database Engineering Team

Table of Contents

1. Introduction.....	1
Document Purpose.....	1
Scope & Applicability.....	2
Document Hierarchy.....	2
Freeze Declaration.....	2
Deliverables.....	3
Schema Statistics.....	4
2. Prisma Standards.....	4
ORM Selection & Version.....	5
ORM Rules.....	5
Prisma Client Usage.....	5
3. Folder Structure.....	10
Multi-File Schema Overview.....	10
Directory Tree.....	10
Folder Organization Rules.....	16
4. Schema Architecture.....	17
Architecture Overview.....	17
Schema Layers.....	17
Multi-Tenant Strategy.....	18
Audit Columns.....	18
Sample Model — Student.....	18
5. Common Models.....	26
Models in this Domain.....	26
Base Mixin (Audit Columns).....	26

6. Identity Models.....	29
Models in this Domain.....	29
Enums Defined.....	29
Sample Model — User.....	29
7. Marketing & CRM Models.....	34
Models in this Domain.....	34
Enums Defined.....	34
Sample Model — Lead.....	34
8. Admissions Models.....	41
Models in this Domain.....	41
Enums Defined.....	41
Admission Workflow.....	41
9. Student Models.....	42
Models in this Domain.....	42
Enums Defined.....	42
Canonical Student Model.....	42
10. Academics Models.....	50
Models in this Domain.....	50
Enums Defined.....	50
Class → Section → Subject Hierarchy.....	50
11. Daily Operations Models.....	50
Models in this Domain.....	51
Enums Defined.....	51
Attendance Lifecycle.....	51
12. Communication Models.....	51
Models in this Domain.....	52
Enums Defined.....	52
Message → Notification Pattern.....	52

13. Finance Models.....	52
Models in this Domain.....	52
Enums Defined.....	53
Immutability Rules.....	53
Invoice Lifecycle.....	53
14. Inventory Models.....	53
Models in this Domain.....	54
Enums Defined.....	54
Stock Movement Pattern.....	54
15. HR Models.....	54
Models in this Domain.....	54
Enums Defined.....	55
Employee ↔ User Linkage.....	55
16. Administration Models.....	55
Models in this Domain.....	55
Enums Defined.....	55
Audit Log Design.....	56
17. Reports Models.....	56
Models in this Domain.....	56
Enums Defined.....	56
Read-Only Projection Layer.....	56
18. Settings Models.....	57
Models in this Domain.....	57
Enums Defined.....	57
3-Level Theme Engine Storage.....	57
19. Platform Models.....	58
Models in this Domain.....	58
Enums Defined.....	58

Multi-Tenant Hierarchy.....	58
Subscription & Plan.....	58
20. Relations.....	59
Overview.....	59
One-to-One Relations.....	59
One-to-Many Relations.....	61
Many-to-Many Relations.....	63
Cascade Rules.....	64
21. Constraints.....	66
Overview.....	67
Primary Keys.....	67
Unique Constraints.....	67
Foreign Keys.....	68
Check Constraints.....	69
Check Constraint Example.....	70
22. Indexes.....	73
Overview.....	73
Index Types.....	73
Indexing Rules.....	73
Composite Index Examples.....	74
Full-Text & GIN Index Example.....	75
23. Enums.....	77
Overview.....	77
Enums by Domain.....	78
Naming Conventions.....	78
Sample Enum Definitions.....	79
24. Migrations.....	85
Overview.....	85

Migration Folder Structure.....	85
Migration Rules.....	88
CI Pipeline Configuration.....	89
Sample Migration.....	94
25. Seed Data.....	103
Overview.....	103
Master Data.....	104
Identity Defaults.....	104
Demo School.....	105
Lookup Tables.....	106
Seed Script Structure.....	108
26. Performance.....	114
Overview.....	115
Connection Pooling.....	115
Query Optimization.....	115
Partitioning.....	116
Read Replicas.....	116
Caching Strategy.....	117
Performance Benchmarks.....	117
Benchmark Verification SQL.....	118
27. Naming Standards.....	120
Overview.....	120
Naming Conventions.....	120
Anti-Patterns.....	120
Schema Validator.....	122
Sign-off & Approval.....	129
Freeze Declaration.....	129
Approval Matrix.....	129

Next Steps.....129

Document Control.....130

Right-click the table above and choose "Update Field" to refresh page numbers.

1. Introduction

Document Purpose

This Prisma Schema v3.0 document is the authoritative reference for the complete database implementation of the PreOne Enterprise Preschool Operating System. It consolidates 288 Prisma models, 84 enum definitions, 412 foreign key relations, full index and constraint specifications, 15 migration scripts, comprehensive seed data, and CI/CD pipeline configuration into a single Database Implementation Freeze. Every NestJS service, every Prisma Client call, and every database migration in the PreOne platform must conform to this document.

This document supersedes the schema snippets scattered across DDD v1.0 and ERD v3.0. Where those documents describe the domain model and entity relationships conceptually, this document specifies the concrete Prisma schema that engineers will implement. It includes the model definitions, the field types, the relation annotations, the index declarations, the constraint definitions, the migration scripts, and the seed data. Engineers should treat this document as the source of truth for any database-related question.

Scope & Applicability

The document covers the entire PostgreSQL-backed data layer for PreOne across all 15 domain bounded contexts: Identity, Marketing & CRM, Admissions, Student, Academics, Daily Operations, Communication, Finance, Inventory, HR, Administration, Reports, Settings, Platform, and Common. It governs schema design, relation cardinalities, multi-tenant isolation, audit columns, soft-delete semantics, indexing strategy, enum taxonomy, migration lifecycle, seed data, performance tuning, and naming conventions. Any engineer, contractor, or AI agent touching the prisma/ directory must read this document first.

The document applies to all PreOne environments: local development, staging, production, and the ephemeral preview environments spun up for each pull request. It applies to the primary PostgreSQL database; it does NOT apply to Redis (cache), Elasticsearch (search indexing), or S3 (file storage) — those have their own schemas documented separately. Any data that needs to be queried relationally, transactionally, or with multi-tenant isolation lives in PostgreSQL and follows this schema.

Document Hierarchy

This document is the successor to DDD v1.0 (domain model), ERD v3.0 (entity-relationship design), and ADR v3.0 (architectural decisions). It is the predecessor to the NestJS Backend implementation, the Prisma Client codegen pipeline, the database migration runner, and the seed scripts. Where this document conflicts with a predecessor, this document takes precedence for database execution. Where the NestJS implementation conflicts with this document, this document wins; the implementation must be corrected.

Document	Version	Relationship	Authority
DDD Document	v1.0	Predecessor	Domain model and bounded contexts
ERD Document	v3.0	Predecessor	Entity-relationship design
ADR Catalog	v3.0	Predecessor	Architectural decisions
** Prisma Schema **	**v3.0**	**This document**	**Database implementation**
NestJS Backend	TBD	Successor	Service implementations
Prisma Client	TBD	Successor	Generated type-safe client
Database Migration	TBD	Successor	Migration runner and CI integration
Seed Scripts	TBD	Successor	Master and demo data seeding

Freeze Declaration

Status: Database Implementation Freeze. The schema in this document is FROZEN. Any change — adding a column, renaming an enum, altering a relation, modifying an index — requires (a) a written change request reviewed by the Architecture Council, (b) an updated ADR documenting the rationale, (c) a new migration script following the rules in Chapter 24, and (d) regeneration of the Prisma Client. Emergency hotfix changes are permitted only for security vulnerabilities and data-loss bugs; all other changes wait for the next minor version (v3.1, v3.2, etc.).

The freeze is enforced by: (a) a branch protection rule on the main branch that requires the architecture council's approval on any PR touching prisma/; (b) a CI check that fails if the prisma migrate diff output is non-empty without a corresponding migration file; (c) a pre-commit hook that validates schema naming conventions and audit-column completeness; and (d) a quarterly architecture review that audits any schema drift since the last review.

Deliverables

This document produces the following production artifacts. Each artifact is independently testable and deployable. The engineering team consumes these artifacts in the order listed below; skipping any step produces an incomplete or non-functional database layer.

- schema.prisma (complete 288 tables, organized via prismaSchemaFolder preview feature)
- Domain-wise Prisma Models (15 domain folders, each with its own .prisma files)
- 84 Enum Definitions (organized by domain, exported from enums/ folder)
- 412 Foreign Key Relations (with explicit @relation annotations and cascade rules)
- Indexes & Constraints (composite indexes, GIN full-text indexes, check constraints)
- 15 Migration Scripts (one per domain, immutable, version-controlled)
- Seed Data Scripts (master data, identity defaults, demo school, lookup tables)
- Prisma Client Generation (npm run prisma:generate produces typed client)
- Database Validation Rules (Zod schemas derived from Prisma types)
- CI/CD Migration Pipeline (GitHub Actions workflow with prisma migrate)

Schema Statistics

The schema is large by any standard. The numbers below reflect the v3.0 freeze and include all 15 domain bounded contexts. Each number is verified by the CI pipeline on every PR; any drift triggers a failure and blocks merge.

Metric	Count	Notes
Prisma Models	288	Across 15 domain folders
Enum Definitions	84	Stored as PostgreSQL ENUM types
Foreign Key Relations	412	With explicit cascade rules

Metric	Count	Notes
Domain Bounded Contexts	15	Identity, CRM, Admissions, etc.
Migration Scripts	15	001_initial through 015_platform
Database Indexes	850	B-tree, GIN, partial, hash
Constraints	612	PK, UK, FK, CHECK
Seed Records	4500	Master + identity + demo school

2. Prisma Standards

ORM Selection & Version

PreOne uses Prisma ORM v5+ as the exclusive database access layer. Direct SQL access via pg, knex, or raw queries is FORBIDDEN in application code; the only exception is migration scripts that need database features Prisma cannot express (e.g., GIN index creation, full-text search triggers). Prisma was chosen over TypeORM and MikroORM because of its generated type-safe client, declarative schema syntax, first-class migration tooling, and excellent PostgreSQL feature coverage.

Prisma was selected after a six-week evaluation period during which the team prototyped the same Student-management module in Prisma, TypeORM, MikroORM, and Kysely. Prisma won on three dimensions: (a) the generated TypeScript client eliminated an entire class of runtime errors that TypeORM's decorator-based approach allowed, (b) the migration tooling produced reviewable SQL that DBAs could sign off on, and (c) the declarative schema syntax was readable enough for product managers to review during sprint planning.

Version: Prisma CLI and @prisma/client are pinned to ^5.18.0 in package.json. Minor version upgrades require running the full migration suite against a shadow database in CI and verifying zero diff. Major version upgrades (v6, v7) require an Architecture Decision Record and a coordinated migration plan; they are not permitted in patch releases.. The CLI, the client, and all @prisma/* packages must be on the same minor version. The engine binary is auto-downloaded by the postinstall script and pinned via the binaryTargets field in generator.prisma.

ORM Rules

The six rules below are non-negotiable. They are enforced by code review, by lint rules (eslint-plugin-prisma), and by Prisma middleware that fails fast on violations. An engineer who breaks these rules in a PR will be asked to refactor before the PR can merge.

Rule	Detail
Use Prisma Client everywhere	All database reads and writes go through the generated Prisma Client. No raw queries in services, no repositories that wrap pg, noKnex usage. Prisma's type safety is the contract between the database and the application.
Use Prisma migrations for schema changes	Never run prisma db push against any environment above local dev. All schema changes go through prisma migrate dev (local) and prisma migrate deploy (CI/staging/prod). Migrations are immutable once merged.
Use transactions for multi-table writes	Any operation that writes to two or more tables must use prisma.\$transaction(). Never write multiple tables in sequence without a transaction; partial writes corrupt data.
Use Prisma's typed include/select	Always specify which relations to load via include or select. Never fetch entire rows with all relations (N+1 problem). Use Prisma's type inference to ensure compile-time safety of relation loading.
Use Prisma middleware for cross-cutting concerns	Tenant isolation, soft-delete filtering, and audit logging are implemented as Prisma middleware/extensions. Engineers must not duplicate these concerns in service code.
Use Prisma's schema preview features judiciously	prismaSchemaFolder is enabled for multi-file schema. MultiSchema is NOT enabled (single schema). FullTextIndex is enabled for PostgreSQL GIN indexes. PostgresExtensions is enabled for uuid-oss and pg_trgm.

Prisma Client Usage

The NestJS service example below demonstrates correct Prisma Client usage: explicit tenantId filter, explicit relation loading via include, transaction for multi-table writes, and proper typing via the generated Prisma types. Every service in the PreOne backend follows this pattern.

```
// NestJS service example — correct Prisma Client usage
```

```

@Injectable()
export class StudentService {
  constructor(private prisma: PrismaService) {}

  async findByAdmissionNo(tenantId: string, admissionNo: string) {
    return this.prisma.student.findFirst({
      where: {
        tenantId, // tenant isolation (auto-injected by
middleware)
        admissionNo, // unique within tenant
        deletedAt: null, // soft-delete filter (auto-injected by
middleware)
      },
      include: {
        profile: true, // explicit relation load
        branch: { select: { name: true } }, // minimal select to avoid over-
fetching
        enrollments: {
          where: { academicYear: { isActive: true } },
          include: { section: { include: { class: true } } },
          take: 1,
        },
      },
    });
  }

  async createWithEnrollment(data: CreateStudentDto) {
    return this.prisma.$transaction(async (tx) => {
      const student = await tx.student.create({ data: { ...data.student } });
      await tx.enrollment.create({ data: { ...data.enrollment, studentId:
student.id } });
      await tx.auditLog.create({ data: { action: 'STUDENT_CREATED', entityId:
student.id, ... } });
      return student;
    });
  }
}

```

3. Folder Structure

Multi-File Schema Overview

PreOne's `prisma/` directory uses the `prismaSchemaFolder` preview feature to split the schema across multiple `.prisma` files organized by domain. Each domain owns its own folder with one or more `.prisma` files. The main `schema.prisma` file contains only the `datasource`, `generator`, and `preview` feature declarations. This structure mirrors the DDD bounded contexts and keeps each domain's schema self-contained for easier review and merge conflict resolution.

The `prismaSchemaFolder` preview feature is enabled in `generator.prisma`. This feature allows the schema to be split across multiple `.prisma` files, each in its own domain folder. Prisma merges these files at generation time into a single logical schema. This structure mirrors the DDD bounded contexts and makes schema review easier — a PR touching only the Finance domain only needs to touch `prisma/finance/`, and reviewers can focus on that folder.

Directory Tree

```
prisma/
├── schema.prisma           # datasource + generator + preview features
├── datasource.prisma       # datasource block (PostgreSQL, env
DATABASE_URL)
├── generator.prisma        # generator client block + custom generators
├── enums/                  # 84 enum definitions organized by domain
│   ├── common.prisma      # Gender, YesNo, StatusType, etc.
│   ├── identity.prisma    # UserRole, PermissionAction, AuthProvider
│   ├── student.prisma     # StudentStatus, BloodGroup, Religion
│   ├── attendance.prisma  # AttendanceStatus, AttendanceType
│   ├── finance.prisma     # InvoiceStatus, PaymentMethod, FeeHead
│   ├── communication.prisma # MessageType, NotificationChannel, Priority
│   └── ... (84 enums across 15 files)
├── common/                 # Base models shared across domains
├── platform/               # Tenant, Branch, Subscription, Plan
├── identity/               # User, Role, Permission, Session, user_role
├── crm/                    # Lead, Campaign, Contact, Activity
├── admission/              # Application, AdmissionForm, Document
├── student/                # Student, StudentProfile, Guardian, Enrollment
├── academics/              # AcademicYear, Class, Section, Subject,
Timetable
├── attendance/             # Attendance, AttendanceLog, HolidayCalendar
├── communication/          # Message, Notification, MessageTemplate,
Broadcast
└── finance/                # Invoice, Receipt, FeeHead, Scholarship, Refund
```

```

├── inventory/           # Item, Stock, PurchaseOrder, Supplier, Issuance
├── hr/                  # Employee, Department, Designation, Leave,
Payroll
├── administration/     # AuditLog, SystemConfig, BackupJob,
ImportJob
├── reports/             # ReportDefinition, ReportSchedule, ReportRun,
Dashboard
├── settings/            # SchoolSetting, BranchSetting, ThemeConfig,
CustomField
├── migrations/          # 15 immutable migration folders
│   ├── 001_initial/
│   ├── 002_identity/
│   └── ... (15 migrations total)
├── seed/                # Seed data scripts (TS)
│   ├── master-data.seed.ts # countries, states, cities, languages, currencies
│   ├── identity.seed.ts    # super admin, roles, permissions, default users
│   ├── school.seed.ts      # demo school, academic year, classes, sections
│   ├── lookup.seed.ts      # blood groups, religion, nationality, fee heads
│   └── index.ts            # orchestrator
├── scripts/             # utility scripts
│   ├── validate-schema.ts  # pre-commit schema validator
│   ├── check-migrations.ts # CI: detect pending migrations
│   ├── generate-client.ts  # custom Prisma Client generation hook
│   └── diff-schema.ts      # diff against production schema

```

Folder Organization Rules

The folder structure is not just cosmetic — it is enforced by tooling. The pre-commit hook (scripts/validate-schema.ts) checks that every .prisma file is in the correct domain folder, that no domain folder contains models from another domain, and that enums are defined in enums/ and imported where needed.

- Each domain folder is owned by exactly one engineering team (see RACI matrix in Chapter 27).
- Cross-domain relations are permitted but must be reviewed by both teams in the PR.
- Enums are defined in enums/ and imported across domain schemas using Prisma's enum import syntax.
- The main schema.prisma file MUST NOT contain model definitions; only datasource + generator + preview.
- All file names use kebab-case; all model names use PascalCase; all field names use camelCase.

- Migration folder names follow NNN_description pattern (e.g., 001_initial, 002_identity).

4. Schema Architecture

Architecture Overview

PreOne's schema architecture is a single-schema PostgreSQL design with multi-tenant isolation enforced at the application layer via Prisma middleware. We do NOT use schema-per-tenant or database-per-tenant isolation because (a) the multi-tenant filter is enforced programmatically in every query, (b) tenant-aware RLS (Row-Level Security) is enabled as a defense-in-depth measure, and (c) the operational cost of per-tenant schemas at 1000+ schools would be prohibitive. Every business table contains `tenantId`, `branchId`, and `academicYearId` columns (where applicable) to enable the cascade filter.

The shared-database pattern was chosen after evaluating three alternatives: schema-per-tenant (operational complexity at 1000+ schemas), database-per-tenant (cost-prohibitive at 1000+ databases), and PostgreSQL RLS-only (too easy to bypass with a missed policy). The shared-database pattern with application-layer enforcement plus RLS as defense-in-depth provides the best balance of cost, complexity, and safety. The Prisma middleware is the primary enforcement; if a developer forgets to filter by `tenantId`, the middleware throws and the request fails.

Schema Layers

The schema is organized into five horizontal layers. Each layer has a clear responsibility and a clear ownership boundary. Layers depend only on lower layers (e.g., Domain depends on Identity and Platform, but Identity does not depend on Domain). This prevents circular dependencies and keeps the migration order deterministic.

Layer	Models	Description
Platform Layer	Tenant, Subscription, Plan, Feature, FeatureFlag, AuditLog, SystemConfig	Cross-cutting models that span all tenants. Managed by the platform team.
Identity Layer	User, Role, Permission, Session, RefreshToken, user_role, role_permission	Authentication and authorization. Shared across all domains; no domain owns identity.
Domain Layer	Student, Admission, Attendance, Finance, Inventory, HR, Academics, Communication	Business-domain models. Each domain is owned by one team; cross-domain relations are explicit.

Layer	Models	Description
Reporting Layer	ReportDefinition, ReportSchedule, ReportRun, Dashboard, Widget, WidgetConfig	Read-only projections of domain data. Reports never own data; they reference domain tables.
Settings Layer	SchoolSetting, BranchSetting, ThemeConfig, CustomField, CustomFieldValue	Per-tenant and per-branch configuration. Stored as JSONB for flexibility.

Multi-Tenant Strategy

Pattern	Shared database, shared schema, tenant-discriminator column
Discriminator	tenantId (UUID, indexed, non-null on all business tables)
Enforcement	Prisma middleware (primary), PostgreSQL RLS (defense-in-depth)
Cascade	tenantId is required on insert. Updates to tenantId are FORBIDDEN (immutable). Branch changes within a tenant are permitted only by Admin role.
Isolation	Each query must filter by tenantId. The Prisma middleware automatically injects tenantId from the request context. Direct prisma.client calls without tenantId in WHERE clause are blocked by middleware and fail at runtime.

Audit Columns

Every model inherits from a Base mixin that adds these 7 columns. The createdBy/updatedBy/deletedBy columns store the user ID (UUID) of the actor. The version column is an integer that increments on every update (optimistic concurrency control).

Soft-delete semantics: deletedAt is nullable. Soft-deleted records have deletedAt != null. Prisma middleware filters out soft-deleted records by default. Hard deletes are FORBIDDEN except for: audit cleanup jobs (records > 7 years old), temporary OTP codes (TTL 10 min), and session cache (TTL 24 h).

The seven required audit columns are: `createdAt`, `createdBy`, `updatedAt`, `updatedBy`, `deletedAt`, `deletedBy`, `version`. The `version` column enables optimistic concurrency control: when an update is issued, Prisma middleware automatically adds a `WHERE version = current_version` clause and increments the version. If the row was modified by another transaction, the update matches zero rows and the service throws a `ConcurrencyError`.

Sample Model — Student

The `Student` model below is the canonical example. It demonstrates all the conventions: UUID primary key, `tenantId` + `branchId` multi-tenant columns, `admissionNo` as tenant-scoped unique field, all seven audit columns, explicit relations with cascade rules, composite unique constraint, single-column indexes, composite indexes, and a GIN full-text index on the name columns. Every model in the schema follows this pattern.

```
// Example: Student model with full audit columns, multi-tenant columns, and indexes
model Student {
  id          String      @id @default(uuid())
  tenantId    String
  branchId    String
  admissionNo String
  firstName   String
  lastName    String?
  dob         DateTime
  gender      Gender
  status      StudentStatus
  admissionDate DateTime @default(now())
  photoUrl    String?
  bloodGroup  BloodGroup?
  religion    String?
  nationality  String?
  remarks     String?
  metadata    Json?

  // Audit columns (inherited from Base mixin in actual code; shown here for clarity)
  createdAt   DateTime @default(now())
  createdBy   String
  updatedAt   DateTime @updatedAt
  updatedBy   String
  deletedAt   DateTime?
  deletedBy   String?
  version     Int       @default(1)

  // Relations
  tenant      Tenant    @relation(fields: [tenantId], references: [id])
  branch      Branch    @relation(fields: [branchId], references: [id])
}
```

```
profile      StudentProfile?
enrollments  Enrollment[]
attendances  Attendance[]
guardians    Guardian[]
invoices     Invoice[]

// Constraints
@@unique([tenantId, admissionNo])
@@unique([tenantId, branchId, admissionNo])

// Indexes
@@index([tenantId])
@@index([branchId])
@@index([tenantId, branchId])
@@index([tenantId, status])
@@index([tenantId, academicYearId])
@@index([createdAt])
@@index([deletedAt])
@@index([firstName, lastName])

// Full-text search index (GIN)
@@index([firstName, lastName], type: Gin)

@@schema("public")
@@map("students")
}
```

5. Common Models

Common models are the shared foundation used by every domain. They include base mixins (audit columns), lookup tables (blood groups, religions, nationalities, relationships, transport types), geographic tables (countries, states, cities), and universal enums (Gender, YesNo, StatusType). These models live in the common/ folder and are imported across the schema.

Models in this Domain

Model	Purpose	Key Fields	Indexes
Country	ISO 3166-1 country master (250 records)	id, iso2, iso3, name, phoneCode, currencyCode, flagEmoji	iso2 (unique), iso3 (unique), name
State	ISO 3166-2 state/region master (4000+ records)	id, countryId, code, name	countryId, code+countryId (unique)
City	City/town master (150000+ records)	id, stateId, name, pincode	stateId, name, pincode
Language	ISO 639-1 language master (184 records)	id, code, name, nativeName	code (unique)
Currency	ISO 4217 currency master (180 records)	id, code, name, symbol, decimalPlaces	code (unique)
TimeZone	IANA time zone master (400+ records)	id, name, utcOffset, dstOffset	name (unique)
BloodGroup	Blood group lookup (8 records: A+, A-, B+, B-, AB+, AB-, O+, O-)	id, code, description	code (unique)
Religion	Religion lookup (12 records: Hindu, Muslim, Christian, Sikh, Jain, Buddhist, Jewish, Parsi, etc.)	id, name	name (unique)

Model	Purpose	Key Fields	Indexes
Relationship	Guardian-student relationship lookup (10 records: Father, Mother, Guardian, Grandparent, Uncle, Aunt, Brother, Sister, Other)	id, name, isPrimary	name (unique)
TransportType	Transport mode lookup (6 records: School Bus, Van, Auto, Car, Walk, Public)	id, name	name (unique)
AuditLog	Universal audit trail for all CRUD operations	id, tenantId, entityType, entityId, action, oldValue, newValue, performedBy, performedAt	tenantId, entityType, entityId, performedAt
SystemConfig	Platform-level key-value configuration	id, key, value, dataType, description	key (unique)
ImportJob	Bulk import job tracking (CSV uploads, etc.)	id, tenantId, fileName, status, totalRows, processedRows, errorRows, startedAt, completedAt	tenantId, status, startedAt
ExportJob	Report/file export job tracking	id, tenantId, exportType, status, fileUrl, startedAt, completedAt	tenantId, status
FileUpload	Pre-signed URL file metadata for S3 uploads	id, tenantId, fileName, fileSize, mimeType, s3Key, s3Bucket, uploadedBy	tenantId, uploadedBy, createdAt

Base Mixin (Audit Columns)

Prisma v5 does not yet support model inheritance (mixins). The audit columns are added to each model manually, but their presence is enforced by a pre-commit hook (scripts/validate-schema.ts) that fails if any business model is missing any of the seven required audit columns. The Prisma middleware auto-populates createdBy/updatedBy from the request context and auto-increments version on update.

```
// common/base.prisma — Base mixin for audit columns (Prisma v5+ supports
mixins via generator)
// In practice, these columns are added to each model manually (Prisma does
not yet support model inheritance).
// The Prisma generator is configured with a custom generator that injects
these into every model.

// Required audit columns (added to EVERY business model):
// createdAt    DateTime  @default(now())
// createdBy    String
// updatedAt    DateTime  @updatedAt
// updatedBy    String
// deletedAt    DateTime?
// deletedBy    String?
// version      Int       @default(1)

// These are enforced by:
// 1. Pre-commit hook (scripts/validate-schema.ts) — fails if any model is
missing audit columns
// 2. Prisma middleware — auto-populates createdBy/updatedBy from request
context,
//   auto-increments version on update, blocks hard delete unless explicitly
whitelisted
// 3. CI check — runs the validator on every PR
```


6. Identity Models

The Identity domain owns authentication and authorization for the entire platform. It is the only domain whose models are NOT scoped by tenantId (with the exception of SchoolUser, which links a User to a Tenant). Identity models are owned by the Platform team and follow stricter change-control rules: any change requires Architecture Council approval.

Models in this Domain

Model	Purpose	Key Fields	Indexes
User	Platform user (unique by email across all tenants)	id, email, phone, passwordHash, mfaSecret, status, lastLoginAt	email (unique), phone, status
SchoolUser	Tenant-scoped user profile (links User to Tenant with role)	id, userId, tenantId, branchId, role, designation, employeeCode	userId+tenantId (unique), tenantId, branchId
Role	Role definition (Platform Admin, School Admin, Principal, Teacher, Parent, Accountant, Receptionist, Librarian)	id, name, code, scope, description	code (unique), scope
Permission	Granular permission (e.g., student.create, fee.refund)	id, code, description, resource, action	code (unique), resource+action (unique)
UserRole	Join table: User ↔ Role (many-to-many)	id, userId, roleId, tenantId, branchId	userId, roleId, userId+roleId+tenantId (unique)
RolePermission	Join table: Role ↔ Permission (many-to-many)	id, roleId, permissionId	roleId, permissionId, roleId+permissionId (unique)
Session	Active user session (JWT refresh token, device info)	id, userId, refreshToken, deviceInfo, ipAddress, expiresAt, revokedAt	userId, refreshToken (unique), expiresAt

Model	Purpose	Key Fields	Indexes
RefreshToken	JWT refresh token storage (rotated on each use)	id, userId, tokenHash, parentTokenHash, expiresAt, usedAt	userId, tokenHash (unique), expiresAt
PasswordReset	Password reset token (TTL 30 min)	id, userId, tokenHash, expiresAt, usedAt	userId, tokenHash (unique), expiresAt
EmailVerification	Email verification token (TTL 24 h)	id, userId, email, tokenHash, verifiedAt, expiresAt	userId, tokenHash (unique), email
MfaChallenge	MFA challenge storage (TOTP, SMS)	id, userId, challengeCode, channel, expiresAt, verifiedAt	userId, expiresAt
AuthProvider	External OAuth provider link (Google, Microsoft, Apple)	id, userId, provider, providerUserId, accessToken, refreshToken, expiresAt	userId, provider+providerUserId (unique)

Enums Defined

Enum	Values
UserRoleScope	PLATFORM TENANT BRANCH
UserStatus	ACTIVE INACTIVE SUSPENDED LOCKED PENDING_VERIFICATION
AuthenticatorType	PASSWORD TOTP SMS EMAIL BIOMETRIC OAUTH_GOOGLE OAUTH_MICROSOFT OAUTH_APPLE
PermissionAction	CREATE READ UPDATE DELETE EXPORT APPROVE REJECT ASSIGN
PermissionResource	STUDENT TEACHER PARENT FEE ATTENDANCE ADMISSION INVOICE RECEIPT REPORT USER ROLE SETTING THEME MESSAGE NOTIFICATION
SessionStatus	ACTIVE EXPIRED REVOKED REPLACED

Sample Model — User

The User model is unique across the entire platform (email is globally unique, not tenant-scoped). This design allows a single user to belong to multiple tenants (e.g., a parent with children in two different PreOne-powered schools). The SchoolUser table links a User to a specific Tenant with a role; a user can have different roles in different tenants.

```
model User {
  id          String      @id @default(uuid())
  email       String      @unique
  phone       String?
  passwordHash String?
  mfaSecret    String?
  mfaEnabled   Boolean     @default(false)
  status       UserStatus  @default(PENDING_VERIFICATION)
  emailVerifiedAt DateTime?
  phoneVerifiedAt DateTime?
  lastLoginAt  DateTime?
  failedLoginAttempts Int    @default(0)
  lockedUntil  DateTime?

  // Audit
  createdAt    DateTime    @default(now())
  updatedAt    DateTime    @updatedAt
  deletedAt    DateTime?
  version      Int          @default(1)

  // Relations
  schoolUsers  SchoolUser[]
  sessions     Session[]
  refreshTokens RefreshToken[]
  passwordResets PasswordReset[]
  emailVerifications EmailVerification[]
  mfaChallenges MfaChallenge[]
  authProviders AuthProvider[]
  auditLogs    AuditLog[]   @relation("UserAuditActor")

  @@index([status])
  @@index([lastLoginAt])
  @@index([deletedAt])
  @@map("users")
}
```

7. Marketing & CRM Models

The CRM domain captures marketing leads, enquiry management, and conversion tracking. It is the entry point for prospective parents before they submit a formal admission application. CRM models are scoped by tenantId and branchId.

Models in this Domain

Model	Purpose	Key Fields	Indexes
Lead	Marketing lead from website, walk-in, call, or referral	id, tenantId, branchId, leadSource, leadStatus, parentName, contactPhone, contactEmail, childName, childDob, interestedClass, assignedTo, firstContactAt, lastContactAt	tenantId, branchId, leadStatus, leadSource, assignedTo, createdAt
Campaign	Marketing campaign (Facebook ads, Google ads, referral, event)	id, tenantId, name, channel, startDate, endDate, budget, status, leadsCount, conversionsCount	tenantId, channel, status, startDate
LeadActivity	Touch-point log per lead (call, email, visit, follow-up)	id, leadId, activityType, notes, scheduledAt, completedAt, performedBy	leadId, activityType, scheduledAt
LeadSource	Lead source lookup (Website, Walk-in, Referral, Facebook, Google Ads, Event, Phone Inquiry, Other)	id, tenantId, name, channel, defaultScore	tenantId, name
LeadScore	Lead quality scoring (1-100 scale based on engagement)	id, leadId, score, scoreBreakdown, calculatedAt	leadId, score, calculatedAt

Model	Purpose	Key Fields	Indexes
Contact	Contact directory (parents, alumni, vendors, partners)	id, tenantId, name, type, email, phone, organization, tags	tenantId, type, email, phone
NewsletterSubscription	Newsletter/opt-in email subscription	id, tenantId, email, source, subscribedAt, unsubscribedAt	tenantId, email, subscribedAt
Referral	Referral tracking (existing parent refers new lead)	id, tenantId, referrerUserId, referredLeadId, referralCode, status, rewardAmount, rewardStatus	tenantId, referrerUserId, status

Enums Defined

Enum	Values
LeadStatus	NEW CONTACTED QUALIFIED NURTURE APPLICATION_STARTED CONVERTED LOST DUPLICATE
LeadSourceChannel	WEBSITE WALK_IN PHONE REFERRAL FACEBOOK GOOGLE_ADS INSTAGRAM EVENT EMAIL SMS PARTNER OTHER
LeadActivityType	CALL EMAIL SMS WHATSAPP VISIT FOLLOW_UP MEETING DEMO DOCUMENT_REQUEST
ContactType	PARENT ALUMNI VENDOR PARTNER STAFF OTHER
ReferralStatus	PENDING ELIGIBLE REWARDED EXPIRED REJECTED

Sample Model — Lead

The Lead model captures prospective parents before they apply for admission. A Lead is converted to an AdmissionApplication via the `leadToApplication()` service method, which copies the parent/child/contact fields and creates an Application record. The Lead remains

in the table with status CONVERTED for reporting (conversion rate by source, by month, by class).

```
model Lead {
  id          String    @id @default(uuid())
  tenantId    String
  branchId    String?
  leadNumber  String
  leadSourceId String
  leadStatus  LeadStatus @default(NEW)
  parentName  String
  contactPhone String
  contactEmail String?
  childName   String?
  childDob    DateTime?
  childGender Gender?
  interestedClass String?
  interestedAcademicYear String?
  notes       String?
  tags        String[]
  assignedToId String?
  score       Int       @default(0)
  convertedAt  DateTime?
  convertedToApplicationId String?

  // Audit
  createdAt    DateTime @default(now())
  createdBy    String
  updatedAt    DateTime @updatedAt
  updatedBy    String
  deletedAt    DateTime?
  version      Int       @default(1)

  // Relations
  tenant      Tenant    @relation(fields: [tenantId], references: [id])
  branch      Branch?   @relation(fields: [branchId], references: [id])
  leadSource  LeadSource @relation(fields: [leadSourceId], references: [id])
  activities  LeadActivity[]
  scores      LeadScore[]

  @@unique([tenantId, leadNumber])
  @@index([tenantId, branchId])
  @@index([tenantId, leadStatus])
  @@index([tenantId, assignedToId])
  @@index([tenantId, createdAt])
  @@index([contactPhone])
  @@index([contactEmail])
  @@map("crm_leads")
}
```


8. Admissions Models

The Admissions domain handles formal admission applications, document collection, interview scheduling, admission committee review, and offer management. Once an application is approved and the admission fee is paid, a Student record is created and the application transitions to CONVERTED status. The Admission model preserves the application history; the Student model is the live entity.

Models in this Domain

Model	Purpose	Key Fields	Indexes
AdmissionApplication	Formal admission application (linked to Lead or direct)	id, tenantId, branchId, applicationNo, leadId, academicYearId, applyingForClass, status, parentName, childName, childDob, childGender, primaryContactPhone, primaryContactEmail, addressLine1, addressLine2, city, state, pincode, countryId, previousSchool, previousClass, siblingName, siblingClass, appliedAt	tenantId, branchId, status, academicYearId, applyingForClass, createdAt
AdmissionForm	Form configuration (custom fields per branch/academic year)	id, tenantId, branchId, academicYearId, formName, sections, isActive, startDate, endDate	tenantId, branchId, academicYearId, isActive

Model	Purpose	Key Fields	Indexes
ApplicationDocument	Documents attached to application (birth cert, Aadhaar, photos, previous marks)	id, applicationId, documentType, fileUrl, fileName, fileSize, mimeType, verified, verifiedBy, verifiedAt, rejectionReason	applicationId, documentType, verified
AdmissionInterview	Interview scheduling (parent + child interaction with principal)	id, applicationId, scheduledAt, duration, interviewers, location, status, feedback, rating	applicationId, scheduledAt, status
AdmissionOffer	Admission offer letter (fee structure, payment deadline, acceptance deadline)	id, applicationId, offerLetterUrl, feeStructure, admissionFeeAmount, paymentDeadline, acceptanceDeadline, status, acceptedAt, declinedAt	applicationId, status, paymentDeadline
AdmissionCommittee	Committee review log (each committee member's vote)	id, applicationId, committeeMemberId, decision, comments, decidedAt	applicationId, committeeMemberId
AdmissionChecklist	Checklist of required steps for admission (form, documents, interview, fee)	id, applicationId, itemType, isCompleted, completedAt, completedBy	applicationId, itemType

Enums Defined

Enum	Values
AdmissionStatus	DRAFT SUBMITTED DOCUMENT_PENDING DOCUMENT_VERIFIED INTERVIEW_SCHEDULED INTERVIEW_COMPLETED UNDER_REVIEW OFFERED ACCEPTED REJECTED WAITLISTED CONVERTED WITHDRAWN
DocumentType	BIRTH_CERTIFICATE AADHAAR PASSPORT PHOTO PREVIOUS_MARKSHEET TRANSFER_CERTIFICATE CHARACTER_CERTIFICATE MEDICAL_CERTIFICATE INCOME_CERTIFICATE CASTE_CERTIFICATE ADDRESS_PROOF OTHER
InterviewStatus	SCHEDULED COMPLETED CANCELLED RESCHEDULED NO_SHOW
OfferStatus	PENDING ACCEPTED DECLINED EXPIRED WITHDRAWN
CommitteeDecision	APPROVE REJECT WAITLIST HOLD

Admission Workflow

The admission workflow has 13 statuses, each with a defined entry and exit condition. The workflow is implemented as a state machine in the AdmissionService; the status field on AdmissionApplication reflects the current state. The status transitions are: DRAFT → SUBMITTED → DOCUMENT_PENDING → DOCUMENT_VERIFIED → INTERVIEW_SCHEDULED → INTERVIEW_COMPLETED → UNDER_REVIEW → OFFERED → ACCEPTED → CONVERTED (or REJECTED, WAITLISTED, WITHDRAWN at various points). Each transition is logged in the AuditLog for compliance.

When an application reaches CONVERTED, the AdmissionService.createStudentFromApplication() method creates a Student record, copies the parent/guardian records, creates an Enrollment for the requested academic year and class, and generates the first invoice (admission fee). All of this happens in a single Prisma transaction to ensure atomicity.

9. Student Models

The Student domain is the core of PreOne. A Student is created when an AdmissionApplication is converted. The Student model holds the live academic and operational record; the AdmissionApplication preserves the pre-admission history. Students are scoped by tenantId and branchId and may be enrolled in one or more academic years.

Models in this Domain

Model	Purpose	Key Fields	Indexes
Student	Live student record (post-admission)	id, tenantId, branchId, admissionNo, firstName, lastName, dob, gender, status, admissionDate, photoUrl, bloodGroup, religion, nationality, motherTongue, aadhaarNo, birthCertificateNo, remarks, metadata	tenantId, branchId, status, admissionNo (unique within tenant), dob, gender
StudentProfile	Detailed profile (one-to-one with Student)	id, studentId, medicalConditions, allergies, medications, emergencyContactName, emergencyContactPhone, pickupAuthorizedPersons, transportType, busRouteId, dietaryPreferences	studentId (unique)

Model	Purpose	Key Fields	Indexes
Guardian	Parent/guardian record (multiple per student)	id, studentId, relationship, name, occupation, education, email, phone, alternatePhone, annualIncome, isPrimary, sameAddressAsStudent	studentId, phone, email, isPrimary
Enrollment	Academic year enrollment (student ↔ academic year ↔ class ↔ section)	id, studentId, academicYearId, classId, sectionId, rollNo, enrollmentStatus, enrollmentDate, exitDate, exitReason	studentId, academicYearId, classId, sectionId, academicYearId+studentId (unique)
StudentDocument	Documents linked to student (post-admission: report cards, certificates, photos)	id, studentId, documentType, fileUrl, fileName, uploadedBy, uploadedAt, notes	studentId, documentType, uploadedAt
StudentHealth	Health record (vaccination, BMI, vision, hearing)	id, studentId, height, weight, bmi, visionLeft, visionRight, hearing, bloodPressure, lastCheckupAt, notes	studentId, lastCheckupAt
StudentAchievement	Extracurricular achievements (sports, arts, academics, competitions)	id, studentId, achievementType, title, description, achievedOn, awardedBy, certificateUrl	studentId, achievementType, achievedOn

Model	Purpose	Key Fields	Indexes
StudentExit	Exit/transfer record (TC issuance, exit interview, fee clearance)	id, studentId, exitType, exitDate, exitReason, transferCertificateNo, destinationSchool, feeClearance, clearanceBy, clearanceAt	studentId, exitDate, exitType

Enums Defined

Enum	Values
StudentStatus	ACTIVE INACTIVE TRANSFERRED GRADUATED ARCHIVED SUSPENDED
BloodGroup	A_POSITIVE A_NEGATIVE B_POSITIVE B_NEGATIVE AB_POSITIVE AB_NEGATIVE O_POSITIVE O_NEGATIVE UNKNOWN
Gender	MALE FEMALE OTHER UNSPECIFIED
EnrollmentStatus	ENROLLED PROMOTED DETAINED TRANSFERRED GRADUATED WITHDRAWN LONG_LEAVE
ExitType	TRANSFER GRADUATION WITHDRAWAL EXPULSION DEATH
AchievementType	ACADEMIC SPORTS ARTS MUSIC DANCE OTHER

Canonical Student Model

The Student model is the most-referenced table in the entire schema. It appears as a foreign key in 38 other tables (Attendance, Invoice, Receipt, Enrollment, Guardian, Health, Achievement, Document, Communication, Report, etc.). The model below is the canonical definition; any deviation in any service must be escalated to the architecture council.

```
model Student {
  id          String    @id @default(uuid())
  tenantId    String
  branchId    String
  admissionNo String
  firstName   String
```

```

lastName      String?
dob            DateTime
gender         Gender
status         StudentStatus @default(ACTIVE)
admissionDate DateTime @default(now())
photoUrl       String?
bloodGroup     BloodGroup?
religion       String?
nationality    String?
motherTongue   String?
aadhaarNo      String?
birthCertificateNo String?
remarks        String?
metadata       Json?

// Audit columns
createdAt      DateTime @default(now())
createdBy      String
updatedAt      DateTime @updatedAt
updatedBy      String
deletedAt      DateTime?
deletedBy      String?
version        Int @default(1)

// Relations
tenant         Tenant @relation(fields: [tenantId], references: [id])
branch         Branch @relation(fields: [branchId], references: [id])
profile        StudentProfile?
enrollments    Enrollment[]
attendances    Attendance[]
guardians      Guardian[]
invoices       Invoice[]
documents      StudentDocument[]
health         StudentHealth?
achievements   StudentAchievement[]
exit           StudentExit?

// Constraints
@@unique([tenantId, admissionNo])

// Indexes
@@index([tenantId])
@@index([branchId])
@@index([tenantId, branchId])
@@index([tenantId, status])
@@index([createdAt])
@@index([deletedAt])
@@index([firstName, lastName], type: Gin)

@@map("students")

```

}

10. Academics Models

The Academics domain covers academic year, classes, sections, subjects, timetables, syllabus, and assessments. It is the structural backbone of the school's daily operations. Most academic models are scoped by tenantId and academicYearId.

Models in this Domain

Model	Purpose	Key Fields	Indexes
AcademicYear	School academic year (e.g., 2026-2027)	id, tenantId, name, startDate, endDate, isActive, isCurrent, termsCount	tenantId, isActive, startDate
AcademicTerm	Terms within academic year (e.g., Term 1, Term 2)	id, academicYearId, name, termNo, startDate, endDate	academicYearId, termNo
Class	Class/grade (Nursery, LKG, UKG, Grade 1-12)	id, tenantId, name, displayName, order, category, isActive	tenantId, order, isActive
Section	Section within a class (A, B, C)	id, classId, name, capacity, classTeacherId, isActive	classId, name (unique within class)
Subject	Subject (Math, English, EVS, etc.)	id, tenantId, name, code, category, isOptional, defaultMarks	tenantId, code (unique), category
ClassSubject	Subjects taught in a class (class ↔ subject many-to-many)	id, classId, subjectId, weeklyPeriods, theoryMarks, practicalMarks, passingMarks	classId, subjectId, classId+subjectId (unique)
TeacherSubject	Teacher assigned to subject for a section	id, teacherId, sectionId, subjectId, academicYearId	teacherId, sectionId, subjectId, academicYearId

Model	Purpose	Key Fields	Indexes
Timetable	Weekly timetable for a section	id, sectionId, academicYearId, dayOfWeek, periodNo, subjectId, teacherId, startTime, endTime, roomNo	sectionId, academicYearId, dayOfWeek, periodNo
HolidayCalendar	School holidays and events	id, tenantId, branchId, academicYearId, date, name, type, description	tenantId, branchId, academicYearId, date, type
Syllabus	Subject-wise syllabus per term	id, classSubjectId, academicYearId, termId, chapterName, description, plannedStartDate, plannedEndDate, completedDate, status	classSubjectId, academicYearId, termId
Assessment	Assessment/exam (unit test, midterm, final)	id, tenantId, academicYearId, name, type, termId, startDate, endDate, maxMarks, subjects	tenantId, academicYearId, type, startDate
AssessmentResult	Student marks per assessment per subject	id, assessmentId, studentId, subjectId, marksObtained, grade, remarks, gradedBy, gradedAt	assessmentId, studentId, subjectId, studentId+assessmentId+subjectId (unique)
ReportCard	Generated report card per student per term	id, studentId, academicYearId, termId, pdfUrl, generatedAt, generatedBy, status	studentId, academicYearId, termId

Enums Defined

Enum	Values
ClassCategory	PRE_PRIMARY PRIMARY MIDDLE SECONDARY SENIOR_SECONDARY
SubjectCategory	CORE LANGUAGE MATHEMATICS SCIENCE SOCIAL ARTS PHYSICAL COMPUTER MORAL OTHER
DayOfWeek	MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY SUNDAY
AssessmentType	UNIT_TEST MID_TERM FINAL QUIZ PROJECT ORAL PRACTICAL SURPRISE_TEST
HolidayType	PUBLIC_HOLIDAY SCHOOL_HOLIDAY FESTIVAL NATIONAL_DAY EXAM_DAY EVENT_DAY OTHER
SyllabusStatus	NOT_STARTED IN_PROGRESS COMPLETED DEFERRED
ReportCardStatus	DRAFT GENERATED PUBLISHED DISTRIBUTED ARCHIVED

Class → Section → Subject Hierarchy

The academic hierarchy is: AcademicYear (e.g., 2026-2027) → Class (e.g., Nursery, LKG, UKG, Grade 1-12) → Section (e.g., A, B, C) → Subject (e.g., English, Math, EVS). A ClassSubject join table maps subjects to classes (a class has multiple subjects; a subject is taught in multiple classes). A TeacherSubject table maps teachers to (section, subject) pairs for a given academic year. The Timetable table holds the weekly schedule per section.

11. Daily Operations Models

The Daily Operations domain covers attendance, daily activities, meal tracking, nap tracking, and incident reporting. These models capture the day-to-day operations of the preschool and feed into the parent app's 'Today's Updates' feature.

Models in this Domain

Model	Purpose	Key Fields	Indexes
Attendance	Daily student attendance (one record per student per day)	id, tenantId, branchId, studentId, academicYearId, date, status, checkInTime, checkOutTime, markedBy, markedAt, remarks	tenantId, branchId, studentId, date, status, academicYearId+studentId+date (unique)
AttendanceLog	Real-time attendance log (for live updates to parent app)	id, attendanceId, eventType, eventTime, recordedBy, location, notes	attendanceId, eventTime
StaffAttendance	Staff/teacher daily attendance	id, tenantId, branchId, staffId, date, status, checkInTime, checkOutTime, markedBy	tenantId, branchId, staffId, date
DailyActivity	Daily activity log per section (rhymes, story, outdoor, art)	id, sectionId, academicYearId, date, activityType, title, description, photos, conductedBy, durationMinutes	sectionId, date, activityType

Model	Purpose	Key Fields	Indexes
MealLog	Meal tracking (breakfast, lunch, snack)	id, studentId, date, mealType, itemsServed, itemsConsumed, quantity, photoUrl, notes	studentId, date, mealType
NapLog	Nap tracking (start time, end time, quality)	id, studentId, date, startTime, endTime, duration, quality, notes	studentId, date
DiaperLog	Diaper change log (for toddler programs)	id, studentId, date, changeTime, type, notes, changedBy	studentId, date
IncidentReport	Incident/injury report (mandatory for parent notification)	id, studentId, date, incidentType, description, severity, actionTaken, parentNotified, notifiedAt, reportedBy, witnesses	studentId, date, incidentType, severity
LeaveRequest	Student leave request (parent applies via app)	id, studentId, startDate, endDate, reason, leaveType, status, approvedBy, approvedAt	studentId, status, startDate
VisitorLog	Visitor entry/exit log	id, tenantId, branchId, visitorName, purpose, contactPhone, hostName, checkInTime, checkOutTime, photoUrl, idProofType, idProofNo	tenantId, branchId, checkInTime

Enums Defined

Enum	Values
AttendanceStatus	PRESENT ABSENT LATE HALF_DAY LEAVE HOLIDAY SUSPENDED
AttendanceType	BIOMETRIC RFID MANUAL APP SELFIE
MealType	BREAKFAST MID_MORNING_SNACK LUNCH AFTERNOON_SNACK EVENING_SNACK DINNER
NapQuality	GOOD AVERAGE POOR DID_NOT_NAP
IncidentType	INJURY ILLNESS BEHAVIORAL ACCIDENT ALLERGIC_REACTION LOST_AND_FOUND OTHER
IncidentSeverity	LOW MEDIUM HIGH CRITICAL
LeaveType	CASUAL SICK MEDICAL FAMILY VACATION BEREAVEMENT OTHER
VisitorStatus	CHECKED_IN CHECKED_OUT OVERSTAY DENIED

Attendance Lifecycle

Attendance is the most-frequently-written table in the schema (50,000+ writes per day per tenant at scale). Each student gets one Attendance record per day. The record is created at check-in (status: PRESENT or LATE), updated at check-out (checkOutTime populated), and may be updated multiple times during the day via AttendanceLog entries (e.g., 'left early for doctor appointment'). The unique index on (academicYearId, studentId, date) prevents duplicate attendance records.

12. Communication Models

The Communication domain covers all messaging and notification channels: in-app messages, push notifications, SMS, email, WhatsApp, broadcasts, and message templates. The domain follows a unified-message-then-channels pattern: a Message is created once, then delivered via one or more NotificationChannels.

Models in this Domain

Model	Purpose	Key Fields	Indexes
Message	Unified message (one record, multiple delivery channels)	id, tenantId, senderId, recipientType, recipientId, subject, body, messageType, priority, templateId, attachments, status, scheduledAt, sentAt	tenantId, senderId, recipientId, status, messageType, createdAt
Notification	Per-channel notification delivery record	id, messageId, channel, recipientAddress, status, providerMessageId, sentAt, deliveredAt, readAt, failureReason	messageId, channel, status, recipientAddress
MessageTemplate	Reusable message templates with variables	id, tenantId, name, subject, body, category, variables, channel, isActive, isSystem	tenantId, category, channel, isActive
Broadcast	Broadcast message to a group (class, branch, all parents)	id, tenantId, senderId, audienceType, audienceId, subject, body, channels, sentAt, recipientsCount, deliveredCount, readCount	tenantId, audienceType, sentAt

Model	Purpose	Key Fields	Indexes
BroadcastRecipient	Per-recipient tracking for broadcasts	id, broadcastId, recipientType, recipientId, status, deliveredAt, readAt	broadcastId, recipientId
Conversation	Two-way conversation thread (parent-teacher, principal-parent)	id, tenantId, participantOneId, participantTwoId, lastMessageAt, lastMessagePreview, unreadCountOne, unreadCountTwo	tenantId, participantOneId, participantTwoId, lastMessageAt
ConversationMessage	Individual message in a conversation	id, conversationId, senderId, body, attachments, sentAt, readAt, editedAt	conversationId, sentAt
NotificationPreference	User's per-channel notification preferences	id, userId, channel, category, isEnabled, quietHoursStart, quietHoursEnd	userId, channel
SmsLog	SMS gateway delivery log (per provider)	id, tenantId, provider, toPhone, message, status, cost, providerMessageId, sentAt, deliveredAt, errorCode	tenantId, status, sentAt
EmailLog	Email delivery log (per provider: SES, SendGrid)	id, tenantId, provider, toEmail, subject, body, status, providerMessageId, sentAt, deliveredAt, openedAt, errorCode	tenantId, status, sentAt

Enums Defined

Enum	Values
MessageType	DIRECT BROADCAST ANNOUNCEMENT REMINDER ALERT NOTIFICATION TRANSACTIONAL
NotificationChannel	IN_APP PUSH SMS EMAIL WHATSAPP WEBHOOK
MessagePriority	LOW NORMAL HIGH URGENT
MessageStatus	DRAFT SCHEDULED QUEUED SENT DELIVERED READ FAILED CANCELLED
BroadcastAudienceType	ALL_PARENTS CLASS_PARENTS BRANCH_PARENTS ALL_STAFF CLASS_TEACHERS BRANCH_STAFF CUSTOM_GROUP
ConversationStatus	ACTIVE ARCHIVED BLOCKED REPORTED

Message → Notification Pattern

The communication domain follows a unified-message-then-channels pattern. A Message is created once (with subject, body, recipients, etc.). The MessageService then creates one Notification per (recipient, channel) pair. This decoupling allows a single message to be delivered via multiple channels (in-app, push, SMS, email) with per-channel status tracking. The Notification table records the delivery status, provider message ID, and failure reason per channel.

13. Finance Models

The Finance domain covers fee heads, fee structures, invoices, receipts, scholarships, refunds, and financial reports. It is the highest-stakes domain in PreOne — any data corruption here directly affects money. All Finance models follow strict immutability rules: invoices and receipts are NEVER updated; corrections happen via credit notes and refund receipts.

Models in this Domain

Model	Purpose	Key Fields	Indexes
FeeHead	Master fee head (tuition, admission, transport, meal, activity, exam, library, late fee)	id, tenantId, name, code, description, category, frequency, isOptional, defaultAmount, taxRate, glCode	tenantId, code (unique), category, frequency
FeeStructure	Class-wise fee structure per academic year	id, tenantId, academicYearId, classId, feeHeadId, amount, frequency, dueDate, lateFeeAmount, lateFeeFrequency	tenantId, academicYearId, classId, feeHeadId
Invoice	Student invoice (one per term/month per fee head)	id, tenantId, branchId, studentId, invoiceNo, invoiceDate, academicYearId, termId, feeHeadId, amount, taxAmount, totalAmount, dueDate, status, paidAmount, balanceAmount	tenantId, branchId, studentId, status, dueDate, invoiceNo (unique)

Model	Purpose	Key Fields	Indexes
InvoiceLineItem	Individual line items in an invoice	id, invoiceId, feeHeadId, description, quantity, rate, amount, taxRate, taxAmount, totalAmount	invoiceId
Receipt	Payment receipt (one per payment, can be partial)	id, tenantId, branchId, studentId, receiptNo, receiptDate, invoiceId, paymentMethod, transactionId, amount, paidBy, paidById, collectedBy, status, notes	tenantId, branchId, studentId, receiptNo (unique), paymentMethod, receiptDate
Payment	Raw payment record (gateway response, before allocation)	id, tenantId, paymentMethod, gateway, gatewayTxnId, amount, currency, status, payerName, payerEmail, payerPhone, paidAt, responsePayload	tenantId, gatewayTxnId (unique), status, paidAt
Scholarship	Scholarship master (merit, sibling, staff ward, financial aid)	id, tenantId, name, code, type, discountType, discountValue, applicableFeeHeads, minPercentage, isActive	tenantId, code (unique), type, isActive
StudentScholarship	Scholarship applied to a student	id, studentId, scholarshipId, academicYearId, approvedBy, approvedAt, validUntil, status	studentId, scholarshipId, academicYearId

Model	Purpose	Key Fields	Indexes
Refund	Refund against a receipt	id, tenantId, receiptId, refundNo, refundDate, amount, reason, approvedBy, approvedAt, status, gatewayRefundId, refundedAt	receiptId, status, refundDate
CreditNote	Credit note against an invoice (for invoice corrections)	id, tenantId, invoiceId, creditNoteNo, creditNoteDate, amount, reason, status, adjustedAgainstReceiptId	invoiceId, status, creditNoteDate
FinancialYear	Financial year (Apr-Mar in India)	id, tenantId, name, startDate, endDate, isActive, isClosed	tenantId, isActive, startDate
JournalEntry	Accounting journal entry (double-entry bookkeeping)	id, tenantId, financialYearId, entryNo, entryDate, description, debitAccount, creditAccount, amount, postedBy, postedAt	tenantId, financialYearId, entryDate

Enums Defined

Enum	Values
FeeHeadCategory	TUITION ADMISSION TRANSPORT MEAL ACTIVITY EXAM LIBRARY LAB SPORTS LATE_FEE OTHER
FeeFrequency	ONE_TIME MONTHLY QUARTERLY HALF_YEARLY ANNUALLY TERM_WISE CUSTOM
InvoiceStatus	DRAFT ISSUED PARTIALLY_PAID PAID OVERDUE CANCELLED WRITTEN_OFF

Enum	Values
PaymentMethod	CASH CHEQUE CARD UPI NET_BANKING WALLET BANK_TRANSFER DD ONLINE OTHER
ReceiptStatus	PENDING COLLECTED BOUNCED CANCELLED REVERSED
PaymentStatus	INITIATED PENDING SUCCESS FAILED REFUNDED DISPUTED
ScholarshipType	MERIT SIBLING STAFF_WARD FINANCIAL_AID EARLY_BIRD REFERRAL SPECIAL OTHER
DiscountType	PERCENTAGE FIXED_AMOUNT FULL_WAIVER
RefundStatus	REQUESTED APPROVED PROCESSED REJECTED CANCELLED
CreditNoteStatus	ISSUED ADJUSTED CANCELLED EXPIRED

Immutability Rules

Invoices and receipts are NEVER updated after issuance. Corrections happen via CreditNote (against an invoice) or Refund (against a receipt). This immutability is enforced at three levels: (a) Prisma middleware blocks UPDATE queries on invoices and receipts tables (only INSERT is allowed for corrections); (b) the service layer exposes only `issueInvoice()`, `cancelInvoice()`, `issueReceipt()`, and `refundReceipt()` methods (no `updateInvoice()` or `updateReceipt()`); (c) the database has a trigger that fails any UPDATE on these tables outside of a maintenance window.

Invoice Lifecycle

The invoice lifecycle is: DRAFT → ISSUED → PARTIALLY_PAID → PAID (or OVERDUE if past `due_date` with unpaid balance, CANCELLED if cancelled before payment, WRITTEN_OFF if the balance is written off after multiple collection attempts). Each transition is logged in AuditLog. The `paidAmount` and `balanceAmount` fields are kept in sync with the sum of receipts; a database trigger verifies the invariant `paid_amount + balance_amount = total_amount` on every receipt insert.

14. Inventory Models

The Inventory domain covers items, stock, suppliers, purchase orders, and issuances. PreOne's inventory is primarily for non-academic items: uniforms, books, stationery, art supplies, cleaning supplies, and food provisions. Stock is tracked per branch.

Models in this Domain

Model	Purpose	Key Fields	Indexes
Item	Inventory item master	id, tenantId, name, sku, description, category, unit, hsnCode, taxRate, salePrice, purchasePrice, reorderLevel, maxLevel, imageUrl	tenantId, sku (unique), category
ItemCategory	Item category (Uniform, Book, Stationery, Art, Cleaning, Food, Sports)	id, tenantId, name, parentId, description	tenantId, parentId
Stock	Current stock per item per branch	id, tenantId, branchId, itemId, quantity, reservedQuantity, availableQuantity, lastReceivedAt, lastIssuedAt, valuationMethod, unitValue	tenantId, branchId, itemId, branchId+itemId (unique)
Supplier	Supplier master	id, tenantId, name, contactPerson, email, phone, address, gstin, pan, paymentTerms, creditLimit, isActive	tenantId, name, gstin (unique), isActive

Model	Purpose	Key Fields	Indexes
PurchaseOrder	Purchase order to supplier	id, tenantId, branchId, poNo, poDate, supplierId, expectedDeliveryDate, status, totalAmount, taxAmount, grandTotal, createdBy, approvedBy	tenantId, branchId, poNo (unique), supplierId, status
PurchaseOrderItem	Line items in a PO	id, purchaseOrderId, itemId, quantity, unitPrice, discountPercent, amount, taxRate, taxAmount, totalAmount, receivedQuantity	purchaseOrderId, itemId
GoodsReceipt	Goods receipt against a PO	id, purchaseOrderId, receiptNo, receiptDate, receivedBy, supplierInvoiceNo, supplierInvoiceDate, status, totalAmount	purchaseOrderId, receiptNo, receiptDate
Issuance	Item issuance to student/staff	id, tenantId, branchId, issuanceNo, issuanceDate, issuedToType, issuedToId, itemId, quantity, unitPrice, amount, issuedBy, purpose	tenantId, branchId, issuanceNo, issuedToId, itemId

Model	Purpose	Key Fields	Indexes
StockAdjustment	Stock adjustment (damage, loss, recount)	id, tenantId, branchId, itemId, adjustmentType, quantity, reason, adjustedBy, adjustedAt, beforeQuantity, afterQuantity	tenantId, branchId, itemId, adjustedAt
StockMovement	Stock movement log (every in/out)	id, tenantId, branchId, itemId, movementType, quantity, referenceType, referenceId, movedAt, movedBy	tenantId, branchId, itemId, movedAt, movementType

Enums Defined

Enum	Values
ItemCategory	UNIFORM BOOK STATIONERY ART_SUPPLIES CLEANING FOOD SPORTS FURNITURE EQUIPMENT OTHER
UnitOfMeasure	PIECE KG LITER METER BOX PACK DOZEN SET ROLL BOTTLE
ValuationMethod	FIFO LIFO WEIGHTED_AVERAGE SPECIFIC
PurchaseOrderStatus	DRAFT PENDING_APPROVAL APPROVED SENT PARTIALLY_RECEIVED RECEIVED CANCELLED CLOSED
GoodsReceiptStatus	PENDING PARTIAL COMPLETED REJECTED
IssuanceType	SALE FREE STAFF_USE DAMAGE LOSS RETURN SAMPLE
StockAdjustmentType	INCREASE DECREASE RECOUNT DAMAGE EXPIRY LOSS THEFT

Enum	Values
StockMovementType	PURCHASE SALE ISSUANCE RETURN ADJUSTMENT TRANSFER SCRAPPED

Stock Movement Pattern

Every stock change is recorded as a StockMovement entry (movementType: PURCHASE, SALE, ISSUANCE, RETURN, ADJUSTMENT, TRANSFER, SCRAPPED). The Stock table holds the current quantity; the StockMovement table is the audit trail. This double-entry pattern ensures that any stock discrepancy can be traced back to its source movement. The StockAdjustment table records manual adjustments with a reason and before/after quantities for auditor review.

15. HR Models

The HR domain covers employees, departments, designations, payroll, leave, attendance (staff), and performance reviews. Employees are linked to Users via the SchoolUser table in the Identity domain. HR models are scoped by tenantId and branchId.

Models in this Domain

Model	Purpose	Key Fields	Indexes
Employee	Employee master record	id, tenantId, branchId, employeeCode, firstName, lastName, dob, gender, email, phone, address, joiningDate, confirmationDate, retirementDate, status, photoUrl, aadhaarNo, panNo, uanNo, esiNo, bankName, bankAccountNo, ifscCode	tenantId, branchId, employeeCode (unique), status, joiningDate
Department	Department master (Academic, Admin, Housekeeping, Transport, etc.)	id, tenantId, name, code, headOfDepartmentId, description, isActive	tenantId, code (unique)
Designation	Designation master (Principal, Vice Principal, Teacher, Asst Teacher, Helper, Driver, etc.)	id, tenantId, name, code, level, minSalary, maxSalary, isActive	tenantId, code (unique), level
EmployeeDepartment	Employee-department mapping (an employee can be in multiple departments)	id, employeeId, departmentId, isPrimary, fromDate, toDate	employeeId, departmentId

Model	Purpose	Key Fields	Indexes
EmployeeDesignation	Employee-designation history (tracks promotions)	id, employeeId, designationId, fromDate, toDate, isActive, changedBy, changedAt	employeeId, designationId, isActive
Payroll	Monthly payroll record per employee	id, tenantId, employeeId, month, year, basicSalary, hra, da, allowances, deductions, epf, esi, taxDeducted, netPay, status, processedAt, processedBy	tenantId, employeeId, month, year, status
PayrollComponent	Earning/deduction components master	id, tenantId, name, code, type, calculationType, defaultAmount, isTaxable, isActive	tenantId, code (unique), type
LeaveRequest	Staff leave request	id, tenantId, employeeId, leaveType, startDate, endDate, days, reason, status, appliedAt, approvedBy, approvedAt	tenantId, employeeId, status, startDate
LeaveBalance	Annual leave balance per employee per type	id, employeeId, leaveType, year, openingBalance, accrued, taken, closingBalance	employeeId, leaveType, year

Model	Purpose	Key Fields	Indexes
PerformanceReview	Annual/quarterly performance review	id, tenantId, employeeId, reviewPeriod, reviewerId, ratings, comments, strengths, improvements, goals, status, submittedAt, reviewedAt	tenantId, employeeId, reviewPeriod
Payslip	Generated payslip per payroll	id, payrollId, pdfUrl, generatedAt, generatedBy, emailedAt, status	payrollId

Enums Defined

Enum	Values
EmployeeStatus	ACTIVE ON_LEAVE SUSPENDED TERMINATED RESIGNED RETIRED PROBATION NOTICE_PERIOD
EmployeeType	FULL_TIME PART_TIME CONTRACT CONSULTANT INTERN TEMPORARY
PayrollStatus	DRAFT PROCESSED APPROVED DISBURSED CANCELLED
PayrollComponentType	EARNING DEDUCTION REIMBURSEMENT BONUS OVERTIME
StaffLeaveType	CASUAL SICK EARNED MATERNITY PATERNITY BEREAVEMENT SABBATICAL UNPAID COMP_OFF
LeaveRequestStatus	PENDING APPROVED REJECTED CANCELLED TAKEN LAPSED
ReviewPeriod	QUARTERLY HALF_YEARLY ANNUAL PROBATION_END SPECIAL
ReviewStatus	DRAFT SUBMITTED REVIEWED ACKNOWLEDGED CLOSED

Employee ↔ User Linkage

Employees are linked to the Identity domain via the SchoolUser table. An Employee has a one-to-one relationship with a SchoolUser (which links to a User). This separation allows an employee to have different roles in different branches (e.g., a teacher who is also a coordinator in another branch) and allows an employee to leave the school without losing their User account (which may be reused if they join another PreOne-powered school).

16. Administration Models

The Administration domain covers system-level operations: audit logs, system configs, backup jobs, import jobs, export jobs, and cron job tracking. These models support the day-to-day administration of the PreOne platform and are owned by the Platform team.

Models in this Domain

Model	Purpose	Key Fields	Indexes
AuditLog	Universal audit trail (all CRUD operations across all domains)	id, tenantId, entityType, entityId, action, oldValue, newValue, performedBy, performedAt, ipAddress, userAgent, requestId	tenantId, entityType, entityId, performedAt, performedBy
SystemConfig	Platform-level configuration (key-value)	id, key, value, dataType, description, category, isEditable, lastEditedBy, lastEditedAt	key (unique), category
BackupJob	Database backup job tracking	id, tenantId, backupType, status, startedAt, completedAt, fileSize, fileUrl, checksum, triggeredBy	tenantId, status, startedAt
RestoreJob	Database restore job tracking (disaster recovery)	id, tenantId, backupJobId, status, startedAt, completedAt, performedBy, notes	tenantId, status

Model	Purpose	Key Fields	Indexes
ImportJob	Bulk import tracking (CSV/Excel)	id, tenantId, fileName, fileType, importType, totalRows, processedRows, successRows, errorRows, status, startedAt, completedAt, triggeredBy, errorReportUrl	tenantId, status, startedAt, importType
ExportJob	Report/file export tracking	id, tenantId, exportType, filters, fileUrl, fileSize, status, startedAt, completedAt, triggeredBy	tenantId, status, startedAt
CronJob	Cron job registry (defined cron schedules)	id, name, description, schedule, command, isActive, lastRunAt, lastRunStatus, nextRunAt	isActive, nextRunAt
CronJobRun	Cron job execution log	id, cronJobId, startedAt, completedAt, status, output, error, durationMs	cronJobId, startedAt, status
FeatureFlag	Feature flag for gradual rollout	id, key, description, isEnabled, rolloutPercent, tenantIds, branchIds, expiresAt	key (unique), isEnabled
MaintenanceWindow	Scheduled maintenance window	id, tenantId, startAt, endAt, description, affectedServices, status, announcedAt	tenantId, startAt, status

Enums Defined

Enum	Values
AuditAction	CREATE UPDATE DELETE RESTORE EXPORT IMPORT LOGIN LOGOUT PERMISSION_CHANGE CONFIG_CHANGE
BackupType	FULL INCREMENTAL DIFFERENTIAL SNAPSHOT
BackupStatus	SCHEDULED IN_PROGRESS COMPLETED FAILED CANCELLED EXPIRED
JobStatus	QUEUED RUNNING COMPLETED FAILED CANCELLED TIMEOUT
ImportType	STUDENT STAFF FEE_STRUCTURE INVOICE ATTENDANCE MARKS PARENT OTHER
ExportType	STUDENT_LIST FEE_REPORT ATTENDANCE_REPORT INVOICE_EXPORT RECEIPT_EXPORT CUSTOM_REPORT OTHER
FeatureFlagStrategy	GLOBAL PERCENTAGE TENANT_LIST BRANCH_LIST USER_LIST

Audit Log Design

The AuditLog table is the universal audit trail. Every CRUD operation on every business table produces an AuditLog entry. The entry captures: entityType (table name), entityId (record UUID), action (CREATE/UPDATE/DELETE/etc.), oldValue (JSON of pre-change state, null for CREATE), newValue (JSON of post-change state, null for DELETE), performedBy (user UUID), performedAt (timestamp), ipAddress, userAgent, and requestId (for tracing). AuditLog records are NEVER deleted except by the 7-year retention job.

17. Reports Models

The Reports domain is a read-only projection layer. Report models do not own business data; they reference domain tables (Student, Invoice, Attendance) and aggregate them. Report definitions are stored in DB; report runs produce files (PDF, Excel, CSV) stored in S3.

Models in this Domain

Model	Purpose	Key Fields	Indexes
ReportDefinition	Report configuration (SQL template, parameters, output format)	id, tenantId, name, code, category, description, sqlTemplate, parameters, outputFormat, schedule, isSystem, isActive	tenantId, code (unique), category, isActive
ReportCategory	Report category master (Finance, Attendance, Admissions, Academics, HR)	id, tenantId, name, code, displayOrder, icon, isActive	tenantId, code (unique)
ReportSchedule	Scheduled report generation (daily/weekly/monthly)	id, reportDefinitionId, tenantId, frequency, recipients, nextRunAt, lastRunAt, parameters, isActive	reportDefinitionId, tenantId, nextRunAt, isActive
ReportRun	Report execution record (per run)	id, reportDefinitionId, tenantId, triggeredBy, status, startedAt, completedAt, fileUrl, fileSize, rowCount, error	reportDefinitionId, tenantId, status, startedAt

Model	Purpose	Key Fields	Indexes
Dashboard	Dashboard definition (collection of widgets)	id, tenantId, name, description, ownerRole, isDefault, isSystem, layout	tenantId, ownerRole, isDefault
Widget	Widget master (chart, table, KPI, gauge)	id, name, code, widgetType, dataSource, queryTemplate, parameters, defaultSize, icon	code (unique), widgetType
DashboardWidget	Widget placement on a dashboard	id, dashboardId, widgetId, position, size, parameters, isVisible	dashboardId, widgetId
SavedReport	User-saved report (custom parameters + filters)	id, reportDefinitionId, userId, tenantId, name, parameters, createdAt, lastRunAt	reportDefinitionId, userId, tenantId

Enums Defined

Enum	Values
ReportCategory	FINANCE ATTENDANCE ADMISSIONS ACADEMICS HR COMMUNICATION INVENTORY ADMIN COMPLIANCE CUSTOM
ReportFormat	PDF EXCEL CSV JSON HTML PNG
ReportScheduleFrequency	DAILY WEEKLY MONTHLY QUARTERLY ANNUALLY ON_DEMAND
ReportStatus	QUEUED RUNNING COMPLETED FAILED CANCELLED
WidgetType	KPI LINE_CHART BAR_CHART PIE_CHART TABLE GAUGE HEATMAP TREND PROGRESS ALERT_LIST RECENT_ACTIVITY

Enum	Values
DashboardOwnerRole	PLATFORM_ADMIN SCHOOL_ADMIN PRINCIPAL TEACHER PARENT ACCOUNTANT

Read-Only Projection Layer

Report models do NOT own business data. They reference domain tables (Student, Invoice, Attendance) and aggregate them at query time. The `ReportDefinition.sqlTemplate` field contains a parameterized SQL template that is executed at report-run time. The result is materialized as a PDF, Excel, or CSV file and stored in S3; the `ReportRun` table records the metadata (file URL, row count, run time, triggered by).

18. Settings Models

The Settings domain covers per-tenant, per-branch, and per-user configuration. It includes the 3-Level Theme Engine configuration (Platform → School ↔ Branch), Design Studio settings, custom fields, and various feature toggles.

Models in this Domain

Model	Purpose	Key Fields	Indexes
SchoolSetting	Tenant-level settings (stored as JSONB for flexibility)	id, tenantId, settings, lastUpdatedBy, lastUpdatedAt	tenantId (unique)
BranchSetting	Branch-level settings (overrides SchoolSetting)	id, tenantId, branchId, settings, lastUpdatedBy, lastUpdatedAt	tenantId, branchId (unique)
ThemeConfig	Theme configuration (logo, colors, font, border radius, layout)	id, tenantId, branchId, theme, themeColor, logoUrl, faviconUrl, fontFamily, borderRadius, layoutMode, sidebarStyle, dashboardLayout, welcomeMessage	tenantId, branchId (unique)
CustomField	Custom field definition per entity (Student, Guardian, Employee)	id, tenantId, entityType, fieldName, fieldType, label, isRequired, isFilterable, options, defaultValue, displayOrder	tenantId, entityType, fieldName
CustomFieldValue	Custom field value per entity instance	id, customFieldId, entityId, value, updatedBy, updatedAt	customFieldId, entityId

Model	Purpose	Key Fields	Indexes
NotificationSetting	Per-tenant notification channel config (SMS gateway, email provider)	id, tenantId, smsProvider, smsApiKey, emailProvider, emailFromName, emailFromAddress, whatsappProvider, whatsappApiKey, isEnabled	tenantId (unique)
PaymentGatewayConfig	Payment gateway configuration (Razorpay, Stripe, PayU)	id, tenantId, provider, apiKey, apiSecret, webhookSecret, isActive, isLiveMode	tenantId, provider, isActive
AcademicYearSetting	Per-academic-year configuration	id, tenantId, academicYearId, workingDays, holidayList, examSchedule, feeDueDates	tenantId, academicYearId (unique)
UserRolePreference	Per-user per-role UI preferences (favorite widgets, dashboard layout)	id, userId, role, preferences, updatedAt	userId, role
LocalizationSetting	Per-tenant language/localization settings	id, tenantId, defaultLanguage, supportedLanguages, dateFormat, timeFormat, currencyFormat, numberFormat, timezone	tenantId (unique)

Enums Defined

Enum	Values
LayoutMode	LIGHT DARK AUTO SYSTEM

Enum	Values
SidebarStyle	EXPANDED COLLAPSED ICON_ONLY AUTO_HIDE
CardStyle	COMFORTABLE COMPACT
CustomFieldType	TEXT NUMBER DATE DATETIME SELECT MULTI_SELECT CHECKBOX RADIO TEXTAREA FILE IMAGE COLOR
EntityType	STUDENT GUARDIAN EMPLOYEE LEAD APPLICATION INVOICE RECEIPT BRANCH OTHER
SmsProvider	TWILIO MSG91 FAST2SMS TEXTLOCAL GUPSHUP NONE
EmailProvider	SES SENDGRID MAILGUN SMTP POSTMARK NONE
PaymentGateway	RAZORPAY STRIPE PAYU CASHFREE PAYTM NONE

3-Level Theme Engine Storage

The ThemeConfig table stores the Level-2 (School) and Level-3 (Branch) theme overrides described in the UI Design Philosophy document. A single row per (tenantId, branchId) pair holds all themeable properties: primary/secondary/accent colors, logo URL, favicon URL, font family, border radius (8/12/16px), layout mode (light/dark/auto), sidebar style, dashboard layout, welcome message, school banner URL. NULL values inherit from the parent level (Platform default).

19. Platform Models

The Platform domain covers multi-tenant infrastructure: tenants (schools), branches (campuses), subscriptions, plans, features, feature flags, and platform-level audit. Platform models are NOT scoped by tenantId — they ARE the tenants themselves.

Models in this Domain

Model	Purpose	Key Fields	Indexes
Tenant	School/organization (top-level entity)	id, name, legalName, code, type, status, contactName, contactEmail, contactPhone, address, city, state, countryId, pincode, gstin, pan, logoUrl, faviconUrl, theme, themeColor, subscriptionPlanId, subscriptionStartAt, subscriptionEndAt, trialEndsAt, maxBranches, maxStudents, maxStaff, isWhiteLabel, parentTenantId	code (unique), status, type, parentTenantId
Branch	School campus/branch	id, tenantId, name, code, branchType, status, address, city, state, countryId, pincode, contactPhone, contactEmail, latitude, longitude, branchHeadId, establishedAt, theme, themeColor, logoUrl, bannerUrl	tenantId, code, tenantId+code (unique), status

Model	Purpose	Key Fields	Indexes
SubscriptionPlan	Subscription plan master (Free, Starter, Pro, Enterprise, White-Label)	id, name, code, description, priceMonthly, priceAnnual, maxBranches, maxStudents, maxStaff, features, isWhiteLabel, isOem, isActive	code (unique), isActive
Subscription	Tenant's active subscription	id, tenantId, planId, status, billingCycle, currentPeriodStart, currentPeriodEnd, amount, currency, autoRenew, paymentMethodId, cancelledAt	tenantId, status, currentPeriodEnd
Feature	Feature master (one per platform capability)	id, name, code, description, category, isPremium, isAddOn, minPlanCode	code (unique), category
PlanFeature	Plan-feature mapping (which plan includes which features)	id, planId, featureId, isEnabled, limits	planId, featureId, planId+featureId (unique)
TenantFeature	Feature flag overrides per tenant (add-ons purchased)	id, tenantId, featureId, isEnabled, expiresAt, purchasedAt	tenantId, featureId, tenantId+featureId (unique)
Invoice	Platform invoice (B2B — PreOne → Tenant for subscription)	id, tenantId, invoiceNo, invoiceDate, subscriptionId, amount, taxAmount, totalAmount, status, dueDate, paidAt, paymentId	tenantId, status, invoiceNo (unique)

Model	Purpose	Key Fields	Indexes
OemPartner	OEM partner (resells PreOne under their brand)	id, name, code, contactName, contactEmail, contactPhone, commissionPercent, contractStartAt, contractEndAt, isActive	code (unique), isActive
OemTenant	OEM-tenant mapping (which tenants belong to which OEM)	id, oemPartnerId, tenantId, brandedAs, customDomain, isActive	oemPartnerId, tenantId

Enums Defined

Enum	Values
TenantType	SCHOOL CHAIN FRANCHISE OEM WHITE_LABEL DEMO TEST
TenantStatus	ACTIVE SUSPENDED TERMINATED TRIAL EXPIRED PENDING_ACTIVATION
BranchType	MAIN SECONDARY FRANCHISE SATELLITE DEMO
BranchStatus	ACTIVE INACTIVE SUSPENDED MERGED CLOSED
SubscriptionPlanCode	FREE STARTER PRO ENTERPRISE WHITE_LABEL
SubscriptionStatus	ACTIVE PAST_DUE CANCELLED EXPIRED TRIALING PENDING
BillingCycle	MONTHLY QUARTERLY HALF_YEARLY ANNUALLY BIENNIALY ONE_TIME
FeatureCategory	CORE ACADEMICS FINANCE COMMUNICATION HR INVENTORY REPORTS INTEGRATIONS WHITE_LABEL ADD_ON

Multi-Tenant Hierarchy

The platform hierarchy is: Tenant (school/organization) → Branch (campus) → [Business Entities]. A Tenant may have one or more Branches; each Branch belongs to exactly one Tenant. The Tenant.parentTenantId field supports OEM hierarchies (an OEM partner has multiple tenants under them). The Tenant.isWhiteLabel flag, when true, hides all PreOne branding from the tenant's UI; the OEM partner's branding is shown instead.

Subscription & Plan

Each Tenant has a Subscription linked to a SubscriptionPlan (Free, Starter, Pro, Enterprise, White-Label). The plan determines maxBranches, maxStudents, maxStaff, and the set of features enabled (via PlanFeature). Tenants can purchase additional features as add-ons (recorded in TenantFeature). The Subscription auto-renews on the currentPeriodEnd date unless cancelled; the billing cycle (monthly, annual) is recorded on the Subscription.

20. Relations

Overview

PreOne's schema contains 412 foreign key relations across 288 models. Relations are categorized by cardinality: one-to-one (1:1), one-to-many (1:N), and many-to-many (M:N). Every relation has an explicit `@relation` annotation with a name (when ambiguous) and explicit cascade rules. Prisma's referential actions (Cascade, Restrict, SetNull, NoAction) are configured per relation based on business semantics.

Every `@relation` annotation in the schema has an explicit `onDelete` clause. The default is Restrict (safer); Cascade is used only for owned child entities (e.g., `StudentProfile` is owned by `Student` and cascades on delete). The cascade rules are reviewed by the architecture council on every PR that adds or modifies a relation.

One-to-One Relations

One-to-one relations are used when an entity has a detailed profile or extension that is loaded rarely. Examples: `Student` ↔ `StudentProfile`, `SchoolSetting` ↔ `Tenant`. The `onDelete` behavior is usually Cascade (the profile is deleted with the parent) or SetNull (the optional relation is cleared when the parent is deleted).

- `Student` ↔ `StudentProfile` (1:1) — `onDelete`: Cascade (profile deleted with student)
- `Student` ↔ `StudentHealth` (1:1) — `onDelete`: Cascade
- `Student` ↔ `StudentExit` (1:1) — `onDelete`: SetNull (exit can be null until exit happens)
- `SchoolSetting` ↔ `Tenant` (1:1) — `onDelete`: Cascade
- `BranchSetting` ↔ `Branch` (1:1) — `onDelete`: Cascade
- `ThemeConfig` ↔ `Branch` (1:1) — `onDelete`: Cascade
- `Payroll` ↔ `Payslip` (1:1) — `onDelete`: Restrict (payslip cannot be deleted while payroll exists)
- `Tenant` ↔ `Subscription` (1:1 active) — `onDelete`: Restrict

One-to-Many Relations

One-to-many relations are the most common cardinality in PreOne. Examples: `Tenant` → `Branches`, `Student` → `Attendances`, `Invoice` → `Receipts`. The `onDelete` behavior is Restrict for entities with financial or legal implications (cannot delete a `Student` with unpaid

invoices) and Cascade for owned child entities (delete a Student's AttendanceLogs when the Student is deleted).

- Tenant → Branches (1:N) — onDelete: Restrict (cannot delete tenant with branches)
- Branch → Students (1:N) — onDelete: Restrict
- Branch → Staff/Employee (1:N) — onDelete: Restrict
- AcademicYear → Enrollments (1:N) — onDelete: Restrict
- Class → Sections (1:N) — onDelete: Restrict (cannot delete class with sections)
- Section → Students (via Enrollment, 1:N) — onDelete: Restrict
- Student → Guardians (1:N) — onDelete: Cascade
- Student → Attendances (1:N) — onDelete: Cascade
- Student → Invoices (1:N) — onDelete: Restrict
- Invoice → Receipts (1:N) — onDelete: Restrict
- Invoice → LineItems (1:N) — onDelete: Cascade
- Lead → LeadActivities (1:N) — onDelete: Cascade
- AdmissionApplication → Documents (1:N) — onDelete: Cascade
- PurchaseOrder → Items (1:N) — onDelete: Cascade
- Employee → Payrolls (1:N) — onDelete: Restrict

Many-to-Many Relations

Many-to-many relations use explicit join tables (no Prisma implicit M:N). The join table has its own id, audit columns, and any relation-specific metadata (e.g., UserRole.tenantId, RolePermission.isEnabled). Explicit join tables are easier to query, easier to extend, and easier to audit than implicit ones.

- User ↔ Role (M:N) via UserRole join table
- Role ↔ Permission (M:N) via RolePermission join table
- Class ↔ Subject (M:N) via ClassSubject join table
- Student ↔ Guardian (M:N via StudentGuardian if a guardian has multiple students)
- Employee ↔ Department (M:N) via EmployeeDepartment join table
- Plan ↔ Feature (M:N) via PlanFeature join table
- Tenant ↔ Feature (M:N) via TenantFeature join table (add-ons)

- Dashboard ↔ Widget (M:N) via DashboardWidget join table
- ReportDefinition ↔ User (M:N) via SavedReport join table
- Conversation ↔ User (M:N) via ConversationParticipant (if group chats)

Cascade Rules

The cascade rules below are the established patterns. New relations must follow these patterns unless an exception is approved by the architecture council.

```
// Cascade rules — set explicitly on every @relation
// Default is Restrict (safer); Cascade only for owned child entities

// Examples:
profile    StudentProfile @relation(fields: [studentId], references: [id],
onDelete: Cascade)
enrollments Enrollment[] // implied: no cascade on parent delete; child
records block delete
guardians  Guardian[]    // Cascade delete guardians when student deleted

auditLogs  AuditLog[]     @relation("UserAuditActor") // Restrict: audit logs
outlive users

// Pattern: child blocks parent delete (Restrict) for entities with financial/legal
implications
tenant     Tenant         @relation(fields: [tenantId], references: [id], onDelete:
Restrict)
branch     Branch         @relation(fields: [branchId], references: [id],
onDelete: Restrict)
student    Student        @relation(fields: [studentId], references: [id],
onDelete: Restrict)
invoice    Invoice         @relation(fields: [invoiceId], references: [id], onDelete:
Restrict)
```

21. Constraints

Overview

Constraints enforce data integrity at the database level. PreOne uses four types: primary keys (always UUID), unique constraints (single and composite), foreign keys (with explicit cascade rules), and check constraints (database-level validation). Constraints are the last line of defense; application-level validation (Zod) is the first.

Constraints are the last line of defense against data corruption. Application-level validation (Zod schemas derived from Prisma types) catches 99% of invalid data; constraints catch the remaining 1% (race conditions, direct SQL access, application bugs). Constraints are NEVER disabled in production.

Primary Keys

Primary key convention: Every model has an `@id` field of type `String` `@id @default(uuid())`. UUIDs are preferred over auto-incrementing integers because (a) they don't leak record counts, (b) they're safe to expose in URLs, (c) they support distributed creation (useful for offline-first mobile apps), and (d) they avoid coordinate-when-migrating issues. The `uuid-oss` PostgreSQL extension is enabled; Prisma's `@default(uuid())` uses `gen_random_uuid()` under the hood.

Unique Constraints

Unique constraints enforce business uniqueness. Single-column unique (e.g., `User.email`) is for globally-unique fields. Composite unique (e.g., `@@unique([tenantId, admissionNo])`) is for tenant-scoped uniqueness. The latter is far more common in a multi-tenant schema.

- Single-column unique: `@unique` (e.g., `User.email`, `Tenant.code`, `Student.admissionNo` within tenant)
- Composite unique: `@@unique([tenantId, admissionNo])` — `admissionNo` unique within a tenant
- Composite unique with branch scope: `@@unique([tenantId, branchId, admissionNo])` — `admissionNo` unique within branch
- Soft-unique: `@@unique([tenantId, email]) WHERE deletedAt IS NULL` — partial unique index (Prisma does not natively support; added via raw SQL in migration)

Foreign Keys

Foreign key rules:

- All FKs are explicit @relation annotations in Prisma
- ON DELETE behavior is explicit per relation (see Chapter 20)
- ON UPDATE defaults to CASCADE (UUIDs never change, so this is moot but explicit)
- Self-referencing FKs (e.g., ItemCategory.parentId, Tenant.parentTenantId) use @relation("SelfReference") to disambiguate

Check Constraints

Check constraints enforce business invariants at the database level. The ten check constraints below are present in every PreOne database. They are added via raw SQL in migrations (Prisma does not natively support check constraints in schema declarations).

- Amount ≥ 0 : All financial amount columns (Invoice.amount, Receipt.amount, FeeStructure.amount) have CHECK (amount ≥ 0)
- Quantity ≥ 0 : All quantity columns (Stock.quantity, PurchaseOrderItem.quantity) have CHECK (quantity ≥ 0)
- Attendance percent ≤ 100 : CHECK (percentage BETWEEN 0 AND 100)
- Student age ≥ 2 years: CHECK (dob $\leq$ CURRENT_DATE - INTERVAL '2 years') — enforced at DB level to catch app bugs
- Student age ≤ 25 years: CHECK (dob $\geq$ CURRENT_DATE - INTERVAL '25 years') — preschool upper bound
- Email format: CHECK (email $\sim$ '^[@]+@[^@]+\.[^@]+\$')
- Phone format (India): CHECK (phone $\sim$ '^[6-9][0-9]{9}\$') — for Indian schools
- Date ranges: startDate $\leq$ endDate on all date-range fields (AcademicYear, Term, Leave, Subscription)
- Invoice balance: CHECK (paid_amount + balance_amount = total_amount) — invariant
- Payroll net pay: CHECK (net_pay = basic + hra + allowances - deductions - epf - esi - tax)

Check Constraint Example

The SQL below shows how check constraints are added in a migration. The `chk_student_dob_min_age` and `chk_student_dob_max_age` constraints enforce the preschool age range (2-25 years). The `chk_invoice_amounts` and `chk_invoice_balance` constraints enforce the financial invariant that `paid + balance = total`.

```
// Migration SQL — adding a check constraint
-- File: prisma/migrations/005_student/migration.sql
ALTER TABLE students
  ADD CONSTRAINT chk_student_dob_min_age
  CHECK (dob <= CURRENT_DATE - INTERVAL '2 years');

ALTER TABLE students
  ADD CONSTRAINT chk_student_dob_max_age
  CHECK (dob >= CURRENT_DATE - INTERVAL '25 years');

ALTER TABLE invoices
  ADD CONSTRAINT chk_invoice_amounts
  CHECK (amount >= 0 AND tax_amount >= 0 AND total_amount = amount +
tax_amount);

ALTER TABLE invoices
  ADD CONSTRAINT chk_invoice_balance
  CHECK (paid_amount + balance_amount = total_amount AND paid_amount
>= 0 AND balance_amount >= 0);
```

22. Indexes

Overview

PreOne uses 850+ indexes across 288 tables. The indexing strategy prioritizes query patterns: every frequently-queried field has an index, every foreign key has an index, and composite indexes cover multi-column WHERE clauses. Full-text search uses GIN indexes on text columns. JSONB columns use GIN indexes for key lookups.

The indexing strategy is reviewed quarterly by the database engineering team. Unused indexes (identified via `pg_stat_user_indexes.idx_scan = 0` over a 30-day window) are candidates for removal. Missing indexes (identified via `pg_stat_statements` showing slow queries) are added in the next migration.

Index Types

Type	Usage	Count
B-tree (default)	Equality and range queries on most columns. Used for <code>tenantId</code> , <code>branchId</code> , <code>status</code> , <code>dates</code> , foreign keys.	780
GIN (Generalized Inverted Index)	Full-text search on <code>student/guardian names</code> , <code>JSONB metadata columns</code> , <code>array columns (tags)</code> .	42
Hash	Equality-only lookups (rare in PreOne; used for some lookup tables).	8
Partial	Indexes with WHERE clauses (e.g., <code>WHERE deletedAt IS NULL</code> for soft-delete queries).	20

Indexing Rules

The seven rules below govern all index creation. New indexes must follow these rules; the pre-commit hook validates that every foreign key has an index and every `tenantId` column has an index.

Rule	Detail
Every foreign key has an index	PostgreSQL does not auto-index FKs. We index every FK to avoid full table scans on joins.
Every tenantId column has an index	Multi-tenant filter is applied on every query. Indexing tenantId is non-negotiable.
Every composite query has a composite index	If a query filters by (tenantId, branchId, status), there's a composite index on those three columns in that order.
Soft-deleted records indexed separately	Most queries filter WHERE deletedAt IS NULL. Partial indexes on this predicate speed up the common case.
Full-text search uses GIN with trigram	Student name search uses pg_trgm GIN index for fuzzy matching (typos, partial names).
JSONB columns indexed with GIN	metadata, settings, preferences columns use GIN jsonb_path_ops index for fast key lookups.
Avoid over-indexing	Each index slows writes. We benchmark write-heavy tables and remove unused indexes (tracked via pg_stat_user_indexes).

Composite Index Examples

The composite indexes below cover the most frequent query patterns. The column order matters: the leftmost column is the most selective filter, and the index is usable only for queries that filter on a prefix of the columns. For example, `@@index([tenantId, branchId, status])` is usable for queries filtering on tenantId alone, tenantId + branchId, or tenantId + branchId + status — but NOT for queries filtering on branchId alone or status alone.

- `@@index([tenantId, branchId])` — most common multi-tenant filter
- `@@index([tenantId, branchId, status])` — filtered listing queries
- `@@index([tenantId, academicYearId])` — academic year-scoped queries
- `@@index([tenantId, branchId, academicYearId])` — full tenant+branch+year scope
- `@@index([studentId, date])` — daily attendance lookup
- `@@index([studentId, academicYearId, termId])` — report card lookup
- `@@index([invoiceId, status])` — invoice status queries

- @@index([createdBy, createdAt]) — audit log actor + time queries

Full-Text & GIN Index Example

The SQL below shows GIN full-text and JSONB indexes. The `pg_trgm` extension enables trigram-based fuzzy matching (handles typos in student name searches). The `jsonb_path_ops` index on the `metadata` column enables fast key lookups in the JSONB column. The partial index on `(tenantId, branchId, status)` WHERE `deleted_at` IS NULL speeds up the most common query (active students in a tenant/branch).

```
-- Migration SQL — GIN full-text index on student names
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE INDEX idx_students_name_trgm
  ON students
  USING gin (first_name gin_trgm_ops, last_name gin_trgm_ops);

-- JSONB GIN index on metadata column
CREATE INDEX idx_students_metadata_gin
  ON students
  USING gin (metadata jsonb_path_ops);

-- Partial index for non-deleted students (90% of queries)
CREATE INDEX idx_students_active
  ON students (tenant_id, branch_id, status)
  WHERE deleted_at IS NULL;
```

23. Enums

Overview

PreOne defines 84 enums organized by domain. Enums are stored as PostgreSQL ENUM types (not strings) for type safety and storage efficiency. Enum values use UPPER_SNAKE_CASE. Enums are defined once in `enums/<domain>.prisma` files and referenced across domain schemas via Prisma's `enum import` (`prismaSchemaFolder` preview feature).

Enums are stored as PostgreSQL ENUM types (not TEXT) for type safety, storage efficiency (4 bytes vs. variable), and validation (invalid values rejected at insert). The trade-off is that adding a value requires a migration (`ALTER TYPE ... ADD VALUE`); removing a value is not directly supported (requires a rename-and-recreate). For v3.0, this trade-off is acceptable; the 84 enums are stable and rarely change.

Enums by Domain

Domain	Count	Examples
Common	8	Gender, YesNo, StatusType, Country, Language, Currency, TimeZone, SortingOrder
Identity	6	UserRoleScope, UserStatus, AuthenticatorType, PermissionAction, PermissionResource, SessionStatus
Marketing & CRM	5	LeadStatus, LeadSourceChannel, LeadActivityType, ContactType, ReferralStatus
Admissions	5	AdmissionStatus, DocumentType, InterviewStatus, OfferStatus, CommitteeDecision
Student	6	StudentStatus, BloodGroup, Gender (shared), EnrollmentStatus, ExitType, AchievementType

Domain	Count	Examples
Academics	7	ClassCategory, SubjectCategory, DayOfWeek, AssessmentType, HolidayType, SyllabusStatus, ReportCardStatus
Daily Operations	8	AttendanceStatus, AttendanceType, MealType, NapQuality, IncidentType, IncidentSeverity, LeaveType, VisitorStatus
Communication	6	MessageType, NotificationChannel, MessagePriority, MessageStatus, BroadcastAudienceType, ConversationStatus
Finance	10	FeeHeadCategory, FeeFrequency, InvoiceStatus, PaymentMethod, ReceiptStatus, PaymentStatus, ScholarshipType, DiscountType, RefundStatus, CreditNoteStatus
Inventory	8	ItemCategory, UnitOfMeasure, ValuationMethod, PurchaseOrderStatus, GoodsReceiptStatus, IssuanceType, StockAdjustmentType, StockMovementType
HR	8	EmployeeStatus, EmployeeType, PayrollStatus, PayrollComponentType, StaffLeaveType, LeaveRequestStatus, ReviewPeriod, ReviewStatus

Domain	Count	Examples
Administration	7	AuditAction, BackupType, BackupStatus, JobStatus, ImportType, ExportType, FeatureFlagStrategy
Reports	6	ReportCategory, ReportFormat, ReportScheduleFrequency, ReportStatus, WidgetType, DashboardOwnerRole
Settings	8	LayoutMode, SidebarStyle, CardStyle, CustomFieldType, EntityType, SmsProvider, EmailProvider, PaymentGateway
Platform	8	TenantType, TenantStatus, BranchType, BranchStatus, SubscriptionPlanCode, SubscriptionStatus, BillingCycle, FeatureCategory

Naming Conventions

Enum naming follows strict conventions. The conventions are enforced by the pre-commit hook (scripts/validate-schema.ts) which parses the .prisma files and validates every enum name and value.

- Enum names use PascalCase (StudentStatus, LeadStatus)
- Enum values use UPPER_SNAKE_CASE (ACTIVE, IN_PROGRESS, PARTIALLY_PAID)
- Enums are immutable once frozen (v3.0 freeze); adding a value requires a new migration and ADR
- Removing a value is FORBIDDEN after freeze; deprecate instead (mark as DEPRECATED_* in code)
- Enums are exported from enums/<domain>.prisma and imported into domain schemas
- All enums have a fallback UNKNOWN or OTHER value where applicable for forward compatibility

Sample Enum Definitions

The enums below are the most-frequently-referenced in the schema. StudentStatus drives the student listing filters; Gender appears in every student-related form; EnrollmentStatus drives the academic year rollover; ExitType drives the transfer certificate workflow.

```
// enums/student.prisma
enum StudentStatus {
  ACTIVE
  INACTIVE
  TRANSFERRED
  GRADUATED
  ARCHIVED
  SUSPENDED
}

enum BloodGroup {
  A_POSITIVE
  A_NEGATIVE
  B_POSITIVE
  B_NEGATIVE
  AB_POSITIVE
  AB_NEGATIVE
  O_POSITIVE
  O_NEGATIVE
  UNKNOWN
}

enum Gender {
  MALE
  FEMALE
  OTHER
  UNSPECIFIED
}

enum EnrollmentStatus {
  ENROLLED
  PROMOTED
  DETAINED
  TRANSFERRED
  GRADUATED
  WITHDRAWN
  LONG_LEAVE
}

enum ExitType {
  TRANSFER
  GRADUATION
  WITHDRAWAL
```

```
EXPULSION  
DEATH  
}
```

24. Migrations

Overview

PreOne uses Prisma Migrate for schema evolution. Migrations are immutable, version-controlled, one-per-domain, and validated in CI before merge. The 15 initial migrations establish the full v3.0 schema. Subsequent migrations (v3.1, v3.2) add columns, indexes, or relations but never drop tables in production without a deprecation period.

Migrations are the **ONLY** way to change the production schema. Direct SQL access to production is **FORBIDDEN** except for emergency hotfixes (which must be retroactively migrated within 24 hours). The migration runner (`npx prisma migrate deploy`) is invoked by the CI/CD pipeline on every deployment to staging and production; it applies pending migrations in order and fails the deployment if any migration fails.

Migration Folder Structure

The 15 initial migrations establish the full v3.0 schema. Each migration is in its own folder with a `migration.sql` file (DDL) and an optional `data.sql` file (DML for seed or data backfill). The folder name follows the `NNN_description` pattern.

```
prisma/
├─ migrations/
│   └─ 001_initial/           # Base tables (countries, languages, audit_log,
system_config)
│   └─ 002_identity/         # User, Role, Permission, Session, user_role,
role_permission
│   └─ 003_crm/              # Lead, Campaign, LeadActivity, LeadSource,
Contact
│   └─ 004_admission/        # AdmissionApplication, AdmissionForm,
ApplicationDocument, Interview, Offer
│   └─ 005_student/          # Student, StudentProfile, Guardian,
Enrollment, StudentDocument
│   └─ 006_academics/        # AcademicYear, Class, Section, Subject,
ClassSubject, Timetable
│   └─ 007_attendance/       # Attendance, AttendanceLog,
HolidayCalendar, DailyActivity
│   └─ 008_communication/    # Message, Notification,
MessageTemplate, Broadcast, Conversation
│   └─ 009_finance/          # FeeHead, FeeStructure, Invoice, Receipt,
Payment, Scholarship
│   └─ 010_inventory/        # Item, Stock, Supplier, PurchaseOrder,
GoodsReceipt, Issuance
│   └─ 011_hr/               # Employee, Department, Designation, Payroll,
LeaveRequest
```


└─ 012_admin/ FeatureFlag	# BackupJob, ImportJob, ExportJob, CronJob,
└─ 013_reports/ ReportRun, Dashboard, Widget	# ReportDefinition, ReportSchedule,
└─ 014_settings/ CustomField	# SchoolSetting, BranchSetting, ThemeConfig,
└─ 015_platform/ Feature, PlanFeature	# Tenant, Branch, Subscription, Plan,

Migration Rules

The nine rules below are non-negotiable. The pre-commit hook validates rule 1 (immutability) by checking that no existing migration file has been modified. CI validates rule 5 (shadow database) by running prisma migrate diff against a freshly-migrated shadow database.

- Migrations are IMMUTABLE. Once merged, a migration file cannot be edited. Fixes go in a new migration.
- Migrations are version-controlled in Git. Every PR with schema changes must include the new migration files.
- One domain per migration. A migration touching both Identity and Finance must be split into two.
- Rollback strategy: every migration has a corresponding down.sql file (Prisma doesn't generate this; we maintain it manually).
- CI validation: prisma migrate diff is run against a shadow database. If diff is non-empty, CI fails.
- Migration names follow NNN_description pattern (e.g., 016_add_transport_tracking).
- Destructive operations (DROP TABLE, DROP COLUMN) require a 2-week deprecation notice and a feature flag.
- Data migrations (DML) are separated from schema migrations (DDL). Data migrations go in scripts/data-migrations/.
- Long-running data migrations (>30s) are run via scripts/ instead of prisma migrate (to avoid CI timeout).

CI Pipeline Configuration

The GitHub Actions workflow below runs on every PR that touches prisma/. It creates a fresh PostgreSQL database, applies all migrations, runs prisma migrate diff to verify the

schema is in sync with the prisma schema declaration, generates the Prisma Client, and runs the TypeScript type-check. If any step fails, the PR is blocked from merge.

```
# .github/workflows/migrate.yml
name: Database Migration Validation

on:
  pull_request:
    paths:
      - 'prisma/**'
      - 'prisma/migrations/**'

jobs:
  validate-migrations:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: postgres
        ports: ['5432:5432']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - name: Create shadow database
        run: PGPASSWORD=postgres psql -h localhost -U postgres -c "CREATE DATABASE preone_shadow;"
      - name: Apply migrations to shadow DB
        run: npx prisma migrate deploy --schema=prisma/schema.prisma
        env:
          DATABASE_URL:
            postgresql://postgres:postgres@localhost:5432/preone_shadow
      - name: Verify schema is in sync with prisma schema
        run: npx prisma migrate diff --from-schema-datasource
            prisma/schema.prisma --to-schema-datamodel prisma/schema.prisma --exit-
            code
        env:
          DATABASE_URL:
            postgresql://postgres:postgres@localhost:5432/preone_shadow
      - name: Generate Prisma Client
        run: npx prisma generate
      - name: Run type-check
        run: npm run typecheck
```

Sample Migration

The migration below creates the students table with all columns, constraints, indexes, foreign keys, and check constraints. It is the canonical example of a PreOne migration file. Every migration in the prisma/migrations/ folder follows this structure.

```
-- prisma/migrations/005_student/migration.sql
-- Student domain — Student, StudentProfile, Guardian, Enrollment,
StudentDocument

CREATE TABLE "students" (
  "id" TEXT NOT NULL,
  "tenant_id" TEXT NOT NULL,
  "branch_id" TEXT NOT NULL,
  "admission_no" TEXT NOT NULL,
  "first_name" TEXT NOT NULL,
  "last_name" TEXT,
  "dob" TIMESTAMP(3) NOT NULL,
  "gender" TEXT NOT NULL,
  "status" TEXT NOT NULL DEFAULT 'ACTIVE',
  "admission_date" TIMESTAMP(3) NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  "photo_url" TEXT,
  "blood_group" TEXT,
  "religion" TEXT,
  "nationality" TEXT,
  "mother_tongue" TEXT,
  "aadhaar_no" TEXT,
  "birth_certificate_no" TEXT,
  "remarks" TEXT,
  "metadata" JSONB,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  "created_by" TEXT NOT NULL,
  "updated_at" TIMESTAMP(3) NOT NULL,
  "updated_by" TEXT NOT NULL,
  "deleted_at" TIMESTAMP(3),
  "deleted_by" TEXT,
  "version" INTEGER NOT NULL DEFAULT 1,

  CONSTRAINT "students_pkey" PRIMARY KEY ("id")
);

-- Unique constraint: admission_no within tenant
CREATE UNIQUE INDEX "students_tenant_id_admission_no_key"
  ON "students" ("tenant_id", "admission_no");

-- Indexes
CREATE INDEX "students_tenant_id_idx" ON "students" ("tenant_id");
```

```

CREATE INDEX "students_branch_id_idx" ON "students" ("branch_id");
CREATE INDEX "students_tenant_id_branch_id_idx" ON "students"
("tenant_id", "branch_id");
CREATE INDEX "students_tenant_id_status_idx" ON "students" ("tenant_id",
"status");
CREATE INDEX "students_created_at_idx" ON "students" ("created_at");
CREATE INDEX "students_deleted_at_idx" ON "students" ("deleted_at");

-- Full-text index (trigram for fuzzy search)
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX "students_name_trgm_idx"
  ON "students" USING gin ("first_name" gin_trgm_ops, "last_name"
gin_trgm_ops);

-- Check constraints
ALTER TABLE "students"
  ADD CONSTRAINT "chk_student_dob_min_age"
  CHECK ("dob" <= CURRENT_DATE - INTERVAL '2 years');

ALTER TABLE "students"
  ADD CONSTRAINT "chk_student_dob_max_age"
  CHECK ("dob" >= CURRENT_DATE - INTERVAL '25 years');

-- Foreign keys
ALTER TABLE "students"
  ADD CONSTRAINT "students_tenant_id_fkey"
  FOREIGN KEY ("tenant_id") REFERENCES "tenants" ("id") ON DELETE
RESTRICT ON UPDATE CASCADE;

ALTER TABLE "students"
  ADD CONSTRAINT "students_branch_id_fkey"
  FOREIGN KEY ("branch_id") REFERENCES "branches" ("id") ON DELETE
RESTRICT ON UPDATE CASCADE;

```

25. Seed Data

Overview

Seed data is the foundational master data required to bootstrap a PreOne environment. It is idempotent: running seed multiple times does not create duplicates. Seed scripts use upsert (where natural key exists) or check-then-create pattern. The seed is split into four categories: master data, identity defaults, demo school, and lookup tables.

Seed data is run automatically on every fresh environment (local dev, preview, staging) via `npm run prisma:seed`. Production seeding is run manually by the DevOps team only during initial onboarding of a new tenant. Seed scripts are idempotent: they use upsert with natural keys (e.g., `countries.iso2`, `blood_groups.code`) so re-running them does not create duplicates.

Master Data

Master data is shared reference data: countries, states, cities, languages, currencies, time zones. This data is identical across all tenants and rarely changes. Updates to master data (e.g., a new country code) are shipped via migration `data.sql` files, not via seed re-runs.

Category	Count	Source	Example
Countries	250	ISO 3166-1	India (IN, IND, +91, INR, )
States	4000	ISO 3166-2	Maharashtra (IN-MH)
Cities	150000	GeoNames	Pune (MH, 411001)
Languages	184	ISO 639-1	English (en, English)
Currencies	180	ISO 4217	Indian Rupee (INR, ₹, 2)
Time Zones	400	IANA	Asia/Kolkata (UTC+5:30)

Identity Defaults

Identity defaults are the platform-level users, roles, and permissions required to bootstrap authentication. The Super Admin user is the only user that exists before any tenant is

created; it can create tenants, assign subscription plans, and impersonate any user. The default roles and permissions are loaded from the `identity.seed.ts` file.

- **Super Admin User:** Email: `superadmin@preone.app`, Role: `PLATFORM_ADMIN`, password from env
- **Default Roles (8):** `PLATFORM_ADMIN`, `SCHOOL_ADMIN`, `PRINCIPAL`, `TEACHER`, `PARENT`, `ACCOUNTANT`, `RECEPTIONIST`, `LIBRARIAN`
- **Default Permissions (120+):** Generated from `PermissionResource × PermissionAction` matrix
- **Role-Permission Mappings:** Each role seeded with default permission set (see `identity.seed.ts`)
- **System User:** Email: `system@preone.app`, used by background jobs and webhooks

Demo School

The demo school (Sunshine Kids Preschool) is a fully-functional tenant with realistic data: 50 students across 8 classes, 8 staff members, fee structures, sample invoices, and 30 days of attendance. The demo school is used for sales demos, QA testing, and engineer onboarding. It is reset nightly via a cron job.

- **Demo Tenant:** Name: Sunshine Kids Preschool, Code: `SUNSHINE-DEMO`, Type: `DEMO`
- **Demo Branch:** Name: Koramangala Branch, Code: `SUN-KRM`
- **Demo Admin User:** Email: `admin@sunshine.demo`, Role: `SCHOOL_ADMIN`
- **Academic Year:** 2026-2027, active, current
- **Classes (8):** PlayGroup, Nursery, LKG, UKG, Grade 1, Grade 2, Grade 3, Grade 4
- **Sections per Class (2):** A, B (with capacity 25 each)
- **Subjects (10):** English, Math, EVS, Hindi, Art, Music, Dance, Sports, Computer, General Knowledge
- **Demo Students (50):** Across all classes with realistic Indian names
- **Demo Staff (8):** 1 Principal, 4 Teachers, 1 Accountant, 1 Receptionist, 1 Helper
- **Fee Structure:** Per class with all fee heads (tuition, admission, transport, meal)
- **Sample Invoices (10):** Mix of paid, unpaid, partially paid, overdue
- **Sample Attendance (30 days):** Last 30 days for all demo students

Lookup Tables

Lookup tables are small reference tables specific to the preschool domain: blood groups, religions, nationalities, relationships, transport types, fee heads, departments, designations. These are seeded once and rarely change. The `lookup.seed.ts` file is the source of truth for these values.

- **Blood Groups (8):** A+, A-, B+, B-, AB+, AB-, O+, O-
- **Religions (12):** Hindu, Muslim, Christian, Sikh, Jain, Buddhist, Jewish, Parsi, Bahai, Other, Prefer Not Say, Unknown
- **Nationalities (250):** All ISO countries
- **Relationships (10):** Father, Mother, Guardian, Grandparent, Uncle, Aunt, Brother, Sister, Stepparent, Other
- **Transport Types (6):** School Bus, Van, Auto, Car, Walk, Public Transport
- **Fee Heads (10):** Tuition, Admission, Transport, Meal, Activity, Exam, Library, Late Fee, Sports, Other
- **Departments (5):** Academic, Administration, Housekeeping, Transport, Accounts
- **Designations (12):** Principal, Vice Principal, Coordinator, Teacher, Asst Teacher, Helper, Driver, Accountant, Receptionist, Librarian, Nurse, Security

Seed Script Structure

The seed orchestrator (`prisma/seed/index.ts`) calls each seed function in order. The order matters: master data must be seeded before identity (because identity references countries for the super admin's address); identity must be seeded before the demo school (because the demo school's admin user references the `SCHOOL_ADMIN` role); lookup must be seeded before the demo school (because the demo school's students reference blood groups and religions).

```
// prisma/seed/index.ts
import { PrismaClient } from '@prisma/client';
import { seedMasterData } from './master-data.seed';
import { seedIdentity } from './identity.seed';
import { seedDemoSchool } from './school.seed';
import { seedLookup } from './lookup.seed';

const prisma = new PrismaClient();

async function main() {
  console.log('Seeding master data...');
```

```

await seedMasterData(prisma);

console.log('Seeding identity (super admin, roles, permissions)...');
await seedIdentity(prisma);

console.log('Seeding lookup tables...');
await seedLookup(prisma);

console.log('Seeding demo school...');
await seedDemoSchool(prisma);

console.log('\u2705 Seed complete!');
}

main()
  .catch((e) => {
    console.error('Seed failed:', e);
    process.exit(1);
  })
  .finally(async () => {
    await prisma.$disconnect();
  });

// Example: seedLookup.ts (idempotent upsert pattern)
export async function seedLookup(prisma: PrismaClient) {
  const bloodGroups = [
    { code: 'A_POSITIVE', description: 'A+' },
    { code: 'A_NEGATIVE', description: 'A-' },
    { code: 'B_POSITIVE', description: 'B+' },
    // ...
  ];

  for (const bg of bloodGroups) {
    await prisma.bloodGroup.upsert({
      where: { code: bg.code },
      update: { description: bg.description },
      create: bg,
    });
  }
}

```


26. Performance

Overview

Database performance is a first-class concern in PreOne. With 1000+ tenants, 50,000+ students per tenant, and real-time parent app queries, performance must be designed in, not bolted on. This chapter covers connection pooling, query optimization, indexing strategy, read replicas, partitioning, and caching.

Performance is measured continuously. The staging environment runs a synthetic benchmark suite every hour (scripts/benchmark.ts) that exercises the top 50 queries with realistic parameters. Results are stored in a Prometheus instance and visualized in Grafana. Regressions of more than 20% trigger a PagerDuty alert to the database engineering team.

Connection Pooling

Pattern	PgBouncer in transaction mode
Pool Size	20 connections per PgBouncer instance, 4 PgBouncer instances per region
Prisma Config	? connection_limit=10&pool_timeout=20&schema=public added to DATABASE_URL
Notes	Prisma's connection pool is per-process. In serverless (Vercel), each Lambda has its own pool, leading to connection exhaustion. PgBouncer in transaction mode solves this by multiplexing.

Query Optimization

The seven rules below govern query optimization. They are enforced by code review and by the prisma-query-analyzer ESLint plugin (which flags N+1 patterns, missing select/include, and offset-based pagination).

- Always specify select or include in Prisma queries — never fetch all columns by default
- Use cursor-based pagination (cursor + take) instead of offset-based (skip + take) for large lists

- Use Prisma's `$queryRaw` for complex aggregations that would require multiple round trips via the client
- Avoid N+1: use `include` or `select` to load relations in a single query
- Use database-level aggregation (`prisma.invoice.aggregate`) instead of fetching all rows and aggregating in JS
- Batch writes: use `prisma.$transaction` with `createMany` for bulk inserts
- Use `EXPLAIN ANALYZE` on slow queries; add indexes based on actual query plans

Partitioning

High-volume tables (`audit_logs`, `attendance`, `notifications`) are partitioned by `tenantId` + time period. Partitioning reduces index size per partition, enables faster scans (only the relevant partition is scanned), and simplifies data retention (old partitions are dropped instead of deleted row-by-row).

Pattern	Declarative partitioning by <code>tenantId</code> for high-volume tables
Tables	<code>audit_logs</code> (partitioned by <code>tenantId</code> + month), <code>attendance</code> (partitioned by <code>tenantId</code> + <code>academic_year</code>), <code>notifications</code> (partitioned by <code>tenantId</code> + month)
Notes	Partitioning is added via raw SQL in migrations (Prisma does not natively support partitioning). The partitions are created automatically via <code>pg_partman</code> extension.

Read Replicas

Read replicas offload read traffic from the primary. The Prisma v5+ `replicaUrls` feature flag routes read queries to replicas automatically; write queries always go to the primary. Replicas have a small replication lag (<1s typically); services that require read-after-write consistency use the primary for the immediately-following read.

Pattern	1 primary + 2 read replicas per region
Prisma Config	Prisma v5+ supports read replicas via the <code>replicaUrls</code> feature flag
Routing	Write queries go to primary; read queries round-robin across replicas

Fallback	If replicas are unavailable, reads fall back to primary
-----------------	---

Caching Strategy

Three cache layers reduce database load. L1 (in-memory) is per-process and has a 5-minute TTL; it is checked first and serves ~30% of cacheable reads. L2 (Redis) is shared across processes and has a 15-minute TTL; it serves ~50% of cacheable reads. L3 (materialized views) is refreshed nightly and serves the heavy dashboard aggregations. Cache invalidation is event-driven: Prisma middleware emits a Redis pub/sub event on every write; subscribers delete the affected cache keys.

- L1: In-memory cache (Node.js process, 5 min TTL) — for lookup tables, currency rates
- L2: Redis (15 min TTL) — for student lists, dashboard widgets, report metadata
- L3: PostgreSQL materialized views — for daily aggregations (refreshed nightly)
- **Invalidation:** Cache is invalidated on write (event-driven via Prisma middleware emitting Redis pub/sub events).

Performance Benchmarks

The benchmarks below are the targets for the top queries. The targets are p95 (95th percentile) latencies measured at peak load (1000 concurrent users per tenant). Queries that exceed their target trigger a performance investigation.

Query	Target (p95)	Notes
Student list (tenantId + branchId + status filter, paginated)	<50ms p95	Composite index on (tenantId, branchId, status)
Attendance lookup (studentId + date)	<5ms p95	Unique index on (studentId, date)
Invoice list (studentId + status)	<30ms p95	Index on (studentId, status)
Dashboard KPI (aggregate over 30 days)	<200ms p95	Materialized view refreshed hourly
Full-text search (student name)	<100ms p95	GIN trigram index

Query	Target (p95)	Notes
Audit log query (entityType + entityId)	<100ms p95	Composite index on (entityType, entityId, performedAt)

Benchmark Verification SQL

The SQL below shows how to verify a query's performance using EXPLAIN ANALYZE. Engineers should run this on any slow query (latency > 100ms) before adding an index or refactoring the query. The EXPLAIN output reveals whether the query is using the expected index or doing a sequential scan.

```
-- Verify query performance (run in psql)
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, first_name, last_name, status
FROM students
WHERE tenant_id = $1
      AND branch_id = $2
      AND deleted_at IS NULL
      AND status = 'ACTIVE'
ORDER BY created_at DESC
LIMIT 50;

-- Expected: Index Scan using students_tenant_id_branch_id_status_idx
-- Target: <50ms execution time
```

27. Naming Standards

Overview

Naming consistency is the foundation of a maintainable schema. PreOne follows a strict naming convention across all 288 tables, 412 relations, 84 enums, and 850+ indexes. The convention is enforced by a pre-commit hook (`scripts/validate-schema.ts`) that fails the commit if any name violates the rules.

Naming consistency is enforced by the pre-commit hook `scripts/validate-schema.ts`. The hook parses every `.prisma` file in the `prisma/` directory, validates model names (PascalCase singular), field names (camelCase), enum values (UPPER_SNAKE_CASE), and table names (snake_case plural via `@@map`). A commit that violates any naming rule is rejected; the engineer must fix the naming before the commit can succeed.

Naming Conventions

The 14 conventions below cover every named entity in the schema. The conventions are derived from PostgreSQL community best practices, Prisma team recommendations, and the PreOne team's experience with the predecessor schema (which had inconsistent naming and was painful to query).

Element	Convention	Examples
Table name (DB)	snake_case, plural	students, guardians, invoices, attendance_logs, audit_logs
Model name (Prisma)	PascalCase, singular	Student, Guardian, Invoice, AttendanceLog, AuditLog
Column name (DB)	snake_case	tenant_id, branch_id, first_name, created_at, deleted_at
Field name (Prisma)	camelCase	tenantId, branchId, firstName, createdAt, deletedAt
Enum name	PascalCase	StudentStatus, InvoiceStatus, AttendanceStatus
Enum value	UPPER_SNAKE_CASE	ACTIVE, IN_PROGRESS, PARTIALLY_PAID

Element	Convention	Examples
Foreign key column	<referenced_table_singular>_id	student_id, tenant_id, branch_id, invoice_id
Foreign key constraint	<table>_<column>_fkey	students_tenant_id_fkey, invoices_student_id_fkey
Unique constraint	<table>_<columns>_key	students_tenant_id_admission_no_key
Index	<table>_<columns>_idx	students_tenant_id_idx, students_tenant_id_branch_id_idx
Check constraint	chk_<table>_<purpose>	chk_students_dob_min_age, chk_invoices_amount_positive
Join table	<entity_a>_<entity_b> (singular_singular)	user_role, role_permission, class_subject
Migration folder	NNN_description (zero-padded, kebab-case)	001_initial, 002_identity, 015_platform
Prisma file	kebab-case or domain name	common.prisma, identity.prisma, student.prisma

Anti-Patterns

The anti-patterns below are explicitly forbidden. The pre-commit hook flags most of them automatically; the rest are caught in code review. Engineers who propose any of these patterns in a PR will be asked to refactor.

- Abbreviations in names (use 'student' not 'stu', 'invoice' not 'inv')
- Plural model names (Student not Students)
- Singular table names (students not student)
- Mixed case in DB names (snake_case only)
- Reserved words as names (avoid 'order', 'user', 'group' — use 'orders', 'users', 'groups' or prefix)
- Prefixes like 'tbl_' or 'sp_' (no Hungarian notation)
- Magic numbers in enum values (use descriptive names: 'STATUS_ACTIVE' not 'STATUS_1')

- Spaces or special characters in names
- CamelCase in column names (snake_case in DB)
- snake_case in Prisma field names (camelCase in Prisma)

Schema Validator

The validator script (scripts/validate-schema.ts) runs as a pre-commit hook. It walks every .prisma file, parses the model/field/enum declarations, and validates the naming conventions. The script is fast (<500ms for the full schema) and provides clear error messages with file/line references.

```
// scripts/validate-schema.ts — runs as pre-commit hook
import { readFileSync, readdirSync } from 'fs';
import { join } from 'path';

const PRISMA_DIR = 'prisma';
const errors: string[] = [];

function validateModelName(name: string, file: string) {
  if (!/^[A-Z][a-zA-Z0-9]*$/.test(name)) {
    errors.push(`${file}: Model name '${name}' must be PascalCase (singular)`);
  }
  if (name.endsWith('s') && !name.endsWith('ss')) {
    errors.push(`${file}: Model name '${name}' should be singular (not '${name}')`);
  }
}

function validateFieldName(name: string, file: string, model: string) {
  if (!/^[a-z][a-zA-Z0-9]*$/.test(name)) {
    errors.push(`${file}.${model}: Field '${name}' must be camelCase`);
  }
}

function validateEnumValue(value: string, file: string, enumName: string) {
  if (!/^[A-Z][A-Z0-9_]*$/.test(value)) {
    errors.push(`${file}.${enumName}: Enum value '${value}' must be UPPER_SNAKE_CASE`);
  }
}

// Walk all .prisma files
function walk(dir: string) {
  for (const entry of readdirSync(dir, { withFileTypes: true })) {
    const full = join(dir, entry.name);
```

```
    if (entry.isDirectory()) walk(full);
    else if (entry.name.endsWith('.prisma')) {
      const content = readFileSync(full, 'utf8');
      // Parse models, fields, enums and validate (simplified — actual parser is
more robust)
      // ...
    }
  }
}

walk(PRISMA_DIR);

if (errors.length > 0) {
  console.error('Naming violations:');
  errors.forEach(e => console.error(' ' + e));
  process.exit(1);
} else {
  console.log('Naming standards: OK');
}
```


Sign-off & Approval

Freeze Declaration

This Prisma Schema v3.0 document is declared FROZEN as of the date of the last signature below. Any modification requires a written change request, an Architecture Decision Record, and re-approval by all signatories. Emergency hotfix changes for security vulnerabilities or data-loss bugs are permitted without full re-approval, but must be retroactively documented within 72 hours. All other changes wait for the next minor version (v3.1, v3.2).

Approval Matrix

The signatures below certify that the signers have reviewed the Prisma Schema v3.0 document, agree with the design decisions, and authorize the database implementation freeze. Once all signatures are obtained, the schema is FROZEN and any change requires a new ADR and re-approval.

Role	Name	Date
Chief Architect	[To be signed]	[To be signed]
VP Engineering	[To be signed]	[To be signed]
Database Lead	[To be signed]	[To be signed]
Backend Lead	[To be signed]	[To be signed]
DevOps Lead	[To be signed]	[To be signed]
Security Lead	[To be signed]	[To be signed]
QA Lead	[To be signed]	[To be signed]

Next Steps

After the freeze is declared, the following steps will be executed in parallel by the respective teams. Each step has an owner and a target completion date. Progress is tracked in the engineering project board.

- NestJS backend team begins implementation against this schema
- Prisma Client is generated via `npm run prisma:generate`
- Migration runner (`npm run prisma:migrate:deploy`) is integrated into CI/CD

- Seed scripts are run against all new environments (dev, staging, prod)
- Schema validation is enforced via pre-commit hook (scripts/validate-schema.ts)
- Performance benchmarks (Chapter 26) are run weekly against staging
- v3.1 planning opens after first 30 days of production use

Document Control

This document is maintained by the PreOne Architecture & Database Engineering Team. Changes require architecture council approval and a new revision history entry. The document is reviewed quarterly and updated as the schema evolves (v3.1, v3.2, etc.). The current version is always available in the PreOne engineering wiki. Questions, suggestions, and deviation requests should be directed to the architecture council via the engineering Slack channel.

Distribution: All PreOne engineers, contractors, and engineering managers. Classification: Internal Engineering Reference — not for external distribution without VP Engineering approval. The document is stored in the engineering wiki (single source of truth) and mirrored to the monorepo /docs/ folder for version control. The wiki version is authoritative when the two diverge.